

Analiza skupa podataka Marketing & Sales

Autori: Danilo Novaković 101/2018, Luka Jevtić 64/2017

1 Uvod

Analiza i predviđanje prihoda od prodaje danas predstavljaju ključni deo uspešnog poslovanja, jer omogućavaju kompanijama da pametnije raspodele marketinške budžete i donose bolje finansijske odluke. Umesto oslanjanja na subjektivne procene i staru praksu, savremeni pristupi koriste podatke i napredne algoritme kako bi se tačno predvideli budući rezultati na osnovu ulaganja, ponašanja kupaca i kanala prodaje.

Za potrebe ovog projekta korišćen je R programski jezik, koji se pokazao kao izvanredan alat za statističku obradu, kreiranje modela mašinskog učenja i vizuelnu analizu podataka, što nam omogućava da na jasan i efikasan način izvedemo ključne poslovne zaključke.

2 Opis podataka

U ovom projektu analizira se skup podataka Marketing & Sales koji obuhvata 60.000 instanci prikupljenih iz poslovanja u MENA regionu (Riyadh, Dubai, Cairo, Abu Dhabi, Jeddah, Doha, Kuwait City). Skup sadrži sveobuhvatne informacije o marketinškim budžetima, troškovima online i offline oglašavanja, kanalima prodaje, segmentima kupaca.

Skup podataka preuzet je sa platforme Kaggle:

Izvor: Marketing i Sales Dataset

(<https://www.kaggle.com/datasets/abdefattahibrahim/marketing-sales-dataset>)

Cilj rada nam je detaljna analiza faktora koji utiču na ostvareni prihod od prodaje (sales_revenue_usd) i razvoj regresionih modela mašinskog učenja za preciznu predikciju prihoda na osnovu marketinških investicija i parametara ponašanja kupaca.

2.1 Učitavanje i dimenzije skupa podataka

Na početku ćemo učitati set podataka

```
dataset=read.csv("marketing_sales_dataset.csv",header = T)
```

```
print(paste("Broj kolona je:", ncol(dataset)))
```

```
[1] "Broj kolona je: 23"
```

```
print(paste("Broj redova je:", nrow(dataset)))
```

```
[1] "Broj redova je: 60000"
```

2.2 Pregled nekoliko redova, naziva feature-a, i njihova struktura

Nakon što smo učitali podatke možemo da proverimo kojih je dimezija set podataka, vidimo da imamo 60 000 vrsta i 23 kolona.

U nastavku smo pomoću funkcije `head` koja izlistava prvih nekoliko vrsta našeg set-a podataka a names nazive feature-a.

```
head(dataset)
```

```
names(dataset)
```

```
[1] "id" "date"
[3] "region" "sales_channel"
[5] "product_category" "customer_segment"
[7] "season" "marketing_budget_usd"
[9] "ad_spend_online_usd" "ad_spend_offline_usd"
[11] "num_promotions" "discount_percentage"
[13] "num_sales_representatives" "customer_age"
[15] "customer_satisfaction_score" "competitor_price_index"
[17] "website_traffic" "conversion_rate"
[19] "email_open_rate" "social_media_followers"
[21] "days_since_last_purchase" "num_previous_purchases"
[23] "sales_revenue_usd"
```

Funkcijom `str` možemo da proverimo kog su tipa naši feature-i i vidimo da ih imamo 23 od čega 4 numeričke (integer - `int`), 9 decimalnih (numeric - `num`) i 6 karakternih (`chr`)

```
str(dataset)
```

```

'data.frame':  60000 obs. of  23 variables:
 $ id                : int  1 2 3 4 5 6 7 8 9 10 ...
 $ date              : chr  "2020-11-12" "2022-07-05" "2020-1
1-11" "2022-10-01" ...
 $ region            : chr  "Riyadh" "Dubai" "Riyadh" "Cairo"
...
 $ sales_channel     : chr  "Retail Store" "Online" "Online"
"Online" ...
 $ product_category  : chr  "Cosmetics" "Cosmetics" "Electron
ics" "Electronics" ...
 $ customer_segment  : chr  "Regular" "New" "Regular" "Regula
r" ...
 $ season            : chr  "Q4" "Q3" "Q4" "Q4" ...
 $ marketing_budget_usd : num  1665 2452 1026 1103 2518 ...
 $ ad_spend_online_usd : num  952 1014 328 628 777 ...
 $ ad_spend_offline_usd : num  327 415 243 205 818 ...
 $ num_promotions     : int  5 4 8 1 6 7 1 1 4 2 ...
 $ discount_percentage : num  23.4 6 11.5 26.7 38.8 39.7 34 28.
4 10.5 25.4 ...
 $ num_sales_representatives : int  24 48 12 32 35 31 44 40 49 4 ...
 $ customer_age       : int  42 24 32 62 23 44 56 74 30 47 ...
 $ customer_satisfaction_score: num  4.5 3.7 2 2.6 3 2.7 3.5 2.6 2.9
4.7 ...
 $ competitor_price_index : num  0.76 1.07 1.18 0.86 1.01 0.81 1.1
9 0.96 1.18 1.1 ...
 $ website_traffic    : int  1952 3185 1304 1609 366639 7947 5
1772 28601 6177 23614 ...
 $ conversion_rate     : num  0.1193 0.0404 0.1459 0.046 0.167
...
 $ email_open_rate     : num  0.041 0.317 0.303 0.114 0.183 ...
 $ social_media_followers : int  2034 2058 417 20618 57116 60085 4
772 222 639396 2512 ...
 $ days_since_last_purchase : num  25 187 139 308 97 202 65 209 256
108 ...
 $ num_previous_purchases : int  6 7 3 6 6 2 9 7 4 9 ...
 $ sales_revenue_usd   : num  3773 2091 6201 4911 7706 ...

```

Sada nakon što smo videli koji sve feature-i postoje u setu podataka možemo da napišemo kratak opis o njima i da lepo to predstavimo tabelarno, za to nam treba da učitamo paket knitr i iskoristimo funkciju kable.

2.3 Pregled i opis svih feature-a

```
opis_kolona=data.frame(  
  Kolona = c(  
    "id", "date", "region", "sales_channel", "product_category",  
    "customer_segment", "season", "marketing_budget_usd", "ad_spend_on  
line_usd",  
    "ad_spend_offline_usd", "num_promotions", "discount_percentage",  
    "num_sales_representatives", "customer_age", "customer_satisfactio  
n_score",  
    "competitor_price_index", "website_traffic", "conversion_rate",  
    "email_open_rate", "social_media_followers", "days_since_last_purc  
hase",  
    "num_previous_purchases", "sales_revenue_usd"  
  ),  
  Opis = c(  
    "Jedinstveni identifikacioni broj transakcije.",  
    "Datum izvršene transakcije.",  
    "Geografski region u okviru MENA tržišta (Riyadh, Dubai, Cairo, Ab  
u Dhabi, Jeddah, Doha, Kuwait City).",  
    "Kanal preko kojeg je obavljena prodaja (npr. Online, Retail Stor  
e).",  
    "Kategorija proizvoda (Cosmetics, Electronics, Food & Beverage, it  
d.).",  
    "Tip ili segment kupca (Regular, New, Corporate, VIP).",  
    "Sezonski period / kvartal u godini (Q1, Q2, Q3, Q4).",  
    "Ukupan opredeljeni marketinški budžet u dolarima.",  
    "Finansijska sredstva uložena u online oglašavanje.",  
    "Finansijska sredstva uložena u offline marketinške kampanje.",  
    "Broj aktivnih promotivnih ponuda ili akcija tokom transakcije.",  
    "Procenat odobrenog popusta .",  
    "Broj angažovanih prodajnih predstavnika.",  
    "Starost kupca u godinama.",  
    "Ocena zadovoljstva kupca na skali od 1 do 5.",  
    "Indeks cene konkurencije u odnosu na naš proizvod.",  
    "Saobraćaj / posete veb-sajtu.",  
    "Stopa konverzije posetilaca u kupce.",  
    "Procenat otvaranja promotivnih mejlova .",  
    "Broj pratilaca brenda na društvenim mrežama.",  
    "Broj proteklih dana od poslednje kupovine istog kupca.",  
    "Ukupan broj prethodnih kupovina koje je kupac ostvario.",  
    "Ostvareni prihod od prodaje u dolarima (ciljna promenljiva - targ  
et)."  
  )  
)  
)
```

```
kable(opis_kolona, col.names = c("Naziv kolone", "Opis promenljive"))
```

Naziv kolone	Opis promenljive
id	Jedinstveni identifikacioni broj transakcije.
date	Datum izvršene transakcije.
region	Geografski region u okviru MENA tržišta (Riyadh, Dubai, Cairo, Abu Dhabi, Jeddah, Doha, Kuwait City).
sales_channel	Kanal preko kojeg je obavljena prodaja (npr. Online, Retail Store).
product_category	Kategorija proizvoda (Cosmetics, Electronics, Food & Beverage, itd.).
customer_segment	Tip ili segment kupca (Regular, New, Corporate, VIP).
season	Sezonski period / kvartal u godini (Q1, Q2, Q3, Q4).
marketing_budget_usd	Ukupan opredeljeni marketinški budžet u dolarima.
ad_spend_online_usd	Finansijska sredstva uložena u online oglašavanje.
ad_spend_offline_usd	Finansijska sredstva uložena u offline marketinške kampanje.
num_promotions	Broj aktivnih promotivnih ponuda ili akcija tokom transakcije.
discount_percentage	Procenat odobrenog popusta .
num_sales_representatives	Broj angažovanih prodajnih predstavnika.
customer_age	Starost kupca u godinama.
customer_satisfaction_score	Ocena zadovoljstva kupca na skali od 1 do 5.
competitor_price_index	Indeks cene konkurencije u odnosu na naš proizvod.
website_traffic	Saobraćaj / posete veb-sajtu.
conversion_rate	Stopa konverzije posetilaca u kupce.
email_open_rate	Procenat otvaranja promotivnih mejlova .
social_media_followers	Broj pratilaca brenda na društvenim mrežama.
days_since_last_purchase	Broj proteklih dana od poslednje kupovine istog kupca.
num_previous_purchases	Ukupan broj prethodnih kupovina koje je kupac ostvario.
sales_revenue_usd	Ostvareni prihod od prodaje u dolarima (ciljna promenljiva - target).

2.4 Deskriptivna statistika i analiza ciljne promenljive

Nakon što smo definisali šta nam predstavlja svaki feature u skupu podataka, možemo da koristimo funkciju `summary()` kako bismo dobili osnovne statističke pokazatelje za sve naše promenljive, ali posebnu pažnju moramo da posvetimo našoj ciljnoj promenljivoj (`sales_revenue_usd`) koja predstavlja ostvareni prihod od prodaje u američkim dolarima. Njena distribucija ključna za kasnije modelovanje

```
summary(dataset)
```

id	date	region	sales_channel
Min. : 1	Length:60000	Length:60000	Length:60000
1st Qu.:15001	Class :character	Class :character	Class :character
Median :30001	Mode :character	Mode :character	Mode :character
Mean :30001			
3rd Qu.:45000			
Max. :60000			

product_category	customer_segment	season	marketing_budget_usd
Length:60000	Length:60000	Length:60000	Min. : 5
Class :character	Class :character	Class :character	1st Qu.: 21
Mode :character	Mode :character	Mode :character	Median : 49
			Mean : 100
			3rd Qu.: 110
			Max. :5000

ad_spend_online_usd	ad_spend_offline_usd	num_promotions	discount_percentage
Min. : 100.1	Min. : 50.03	Min. :0.000	Min. : 0.0
1st Qu.: 816.8	1st Qu.: 486.86	1st Qu.:2.000	1st Qu.:10.1
Median : 1876.4	Median : 1152.39	Median :4.000	Median :20.0
Mean : 4018.4	Mean : 2508.81	Mean :4.493	Mean :20.0
3rd Qu.: 4336.9	3rd Qu.: 2676.80	3rd Qu.:7.000	3rd Qu.:30.0
Max. :235211.8	Max. :138327.25	Max. :9.000	Max. :40.0

num_sales_representatives	customer_age	customer_satisfaction_score
Min. : 1.00	Min. :18.0	Min. :2.000
1st Qu.:13.00	1st Qu.:31.0	1st Qu.:2.700
Median :25.00	Median :46.0	Median :3.500
Mean :24.93	Mean :45.8	Mean :3.496
3rd Qu.:37.00	3rd Qu.:60.0	3rd Qu.:4.200
Max. :49.00	Max. :74.0	Max. :5.000
		NA's :1844

competitor_price_index	website_traffic	conversion_rate	email_open
------------------------	-----------------	-----------------	------------

```

_rate
Min.   :0.70           Min.    :   100   Min.   :0.00000   Min.   :0.
0024
1st Qu.:0.85           1st Qu.:  2961   1st Qu.:0.04580   1st Qu.:0.
1458
Median :1.00           Median :   8065   Median :0.07890   Median :0.
2166
Mean   :1.00           Mean    :  24426   Mean   :0.09093   Mean   :0.
2308
3rd Qu.:1.15           3rd Qu.: 22057   3rd Qu.:0.12323   3rd Qu.:0.
3009
Max.   :1.30           Max.    :2000000   Max.   :0.53470   Max.   :0.
7537

                                         NA's    :17
90
social_media_followers days_since_last_purchase num_previous_purchase
s
Min.   :   100           Min.    :  0.0           Min.   : 0.000
1st Qu.:   771           1st Qu.: 89.0           1st Qu.: 3.000
Median :  2971           Median :182.0           Median : 5.000
Mean   :  22198          Mean    :181.5           Mean   : 4.985
3rd Qu.: 11387           3rd Qu.:273.0           3rd Qu.: 6.000
Max.   :5000000          Max.    :364.0           Max.   :17.000
NA's   :1836

sales_revenue_usd
Min.   :   481.1
1st Qu.:  2744.2
Median : 4340.1
Mean   :  5911.1
3rd Qu.:  7091.6
Max.   :190377.4

```

Što se tiče naše ciljne promenljive (sales_revenue_usd), iz priloženih statistika se lepo vidi da je prosečna vrednost od oko 5.911 dolara viša od medijane koja iznosi 4.340 dolara. To nam odmah govori da imamo nekoliko transakcija sa prilično velikim prihodima koje vuku prosek nagore. Većina prodaja se vrti oko nižih i srednjih iznosa (pa nam se tako 75% podataka nalazi ispod 7.091 dolar), dok samo manji broj transakcija ide u neke ekstremnije visine, sve do maksimalnih 190.377 dolara.

2.5 Provera nedostajućih vrednosti

Hoćemo da identifikujemo nedostajuće vrednosti možemo da koristimo funkciju colSums i vidimo da email_open_rate ima 1790, discount_percentage ima 1808 ,customer_satisfaction_score ima 1844 i days_since_last_purchaseima 1836 nedostajućih vrednosti.

```
colSums(is.na(dataset))
```

	id		date
region			
0	0		0
	sales_channel	product_category	cus
tomor_segment			
0	0		0
	season	marketing_budget_usd	ad_spe
nd_online_usd			
0	0		0
	ad_spend_offline_usd	num_promotions	discou
nt_percentage			
0	0		0
1808			
	num_sales_representatives	customer_age	customer_satis
faction_score			
0	0		0
1844			
	competitor_price_index	website_traffic	co
nversion_rate			
0	0		0
	email_open_rate	social_media_followers	days_since_
last_purchase			
1790			0
1836			
	num_previous_purchases	sales_revenue_usd	
0	0		0

2.6 Provera duplikata i unikatnih vrednosti

U ovom koraku ćemo proveriti da li u našem setu podataka postoje duplikati, odnosno da li se neki redovi u potpunosti ponavljaju. I vidmo na osnovu rezultata da ih nema i da je svaka od 60.000 instanci jedinstvena.

```
total_duplicate_rows = sum(duplicated(dataset))
print(paste("Ukupan broj dupliranih redova u dataset-u:", total_duplicate_rows))
```

```
[1] "Ukupan broj dupliranih redova u dataset-u: 0"
```

Sada ćemo proveriti koje sve unikatne vrednosti imamo u našim kategorijskim kolonama, da vidimo da li tu ima nekih čudnih unosa ili slovnihih grešaka. Za to možemo da pregledamo unikatne vrednosti za ključne tekstualne promenljive:


```
sapply(dataset[sapply(dataset, is.character)], function(x) length(unique(x)))
```

```

      date      region  sales_channel product_category customer_segment
1461         7         5              5
4
      season
4

```

Vidimo da date ima 1461, region ima 7, sales_channel ima 5, product_category ima 5, customer_segment ima 4 i season ima 4 jedinstvene vrednosti

U ovom koraku ćemo proveriti kako su podaci raspoređeni po kategorijama, odnosno kolika je frekvencija opservacija u ključnim kategorijskim kolonama:

```
table(dataset$region)
```

```

Alexandria    Amman    Cairo Casablanca    Dubai    Kuwait
Riyadh
  9027    4206    14984    4183    10890    4739
11971

```

```
table(dataset$sales_channel)
```

```

Direct Sales    Online Retail Store Social Media    Wholesale
  9209    18010    14954    5840    11987

```

```
table(dataset$product_category)
```

```

Clothing    Cosmetics    Electronics Food & Beverage Home Appliances
  12016    9068    13118    15023
10775

```

```
table(dataset$customer_segment)
```

```

Corporate    New    Regular    VIP
  9113    18011    23906    8970

```

```
table(dataset$season)
```

```
   Q1   Q2   Q3   Q4  
14898 14954 15124 15024
```

Kao što vidimo iz tabela frekvencija, opservacije su prilično ravnomerno raspoređene po različitim regionima, kanalima prodaje, kategorijama proizvoda i sezonama, što nam govori da nemamo ekstremne disbalanse u uzorku.

2.7 Provera validnosti opsega i logičkih pravila

U ovom koraku ćemo proveriti opsege minimalnih i maksimalnih vrednosti za sve numeričke promenljive, kako bismo utvrdili osnovne granice u kojima se podaci kreću:

```
sapply(dataset[sapply(dataset, is.numeric)], function(x) c(Min = min  
(x, na.rm = TRUE), Max = max(x, na.rm = TRUE)))
```

```
      id marketing_budget_usd ad_spend_online_usd ad_spend_offline_us  
d  
Min      1                5e+02                100.14                50.0  
3  
Max 60000                5e+05                235211.83                138327.2  
5  
      num_promotions discount_percentage num_sales_representatives custo  
mer_age  
Min              0              0              1  
18  
Max              9              40              49  
74  
      customer_satisfaction_score competitor_price_index website_traffic  
Min              2              0.7              1e+02  
Max              5              1.3              2e+06  
      conversion_rate email_open_rate social_media_followers days_since_  
last_purchase  
Min              0.0000              0.0024              1e+02  
0  
Max              0.5347              0.7537              5e+06  
364  
      num_previous_purchases sales_revenue_usd  
Min              0              481.08  
Max              17              190377.35
```

Nakon provere opsega, analizirane su minimalne i maksimalne vrednosti da se utvrdi da li ima nekih nelogičnosti. Uočeno je da se svi podaci kreću u normalnim granicama: godine kupaca su od 18 do 74, ocene zadovoljstva su između 2 i 5, a maksimalni popust je 40%. Budžeti i troškovi jesu visoki, ali su u skladu sa realnim poslovanjem i nema negativnih cena niti nelogičnih unosa. Samim tim, numerički opsezi su potpuno ispravni.

U ovom koraku ćemo proveriti da li u setu podataka postoje logičke nepravilnosti:

Provera da li popust prelazi 100% (> 1) ili je negativan

```
sum(dataset$discount_percentage < 0 | dataset$discount_percentage > 100, na.rm = TRUE)
```

```
[1] 0
```

Provera da li ocene zadovoljstva izlaze van opsega [1, 5]

```
sum(dataset$customer_satisfaction_score < 1 | dataset$customer_satisfaction_score > 5, na.rm = TRUE)
```

```
[1] 0
```

Provera da li broj dana od poslednje kupovine prelazi 365

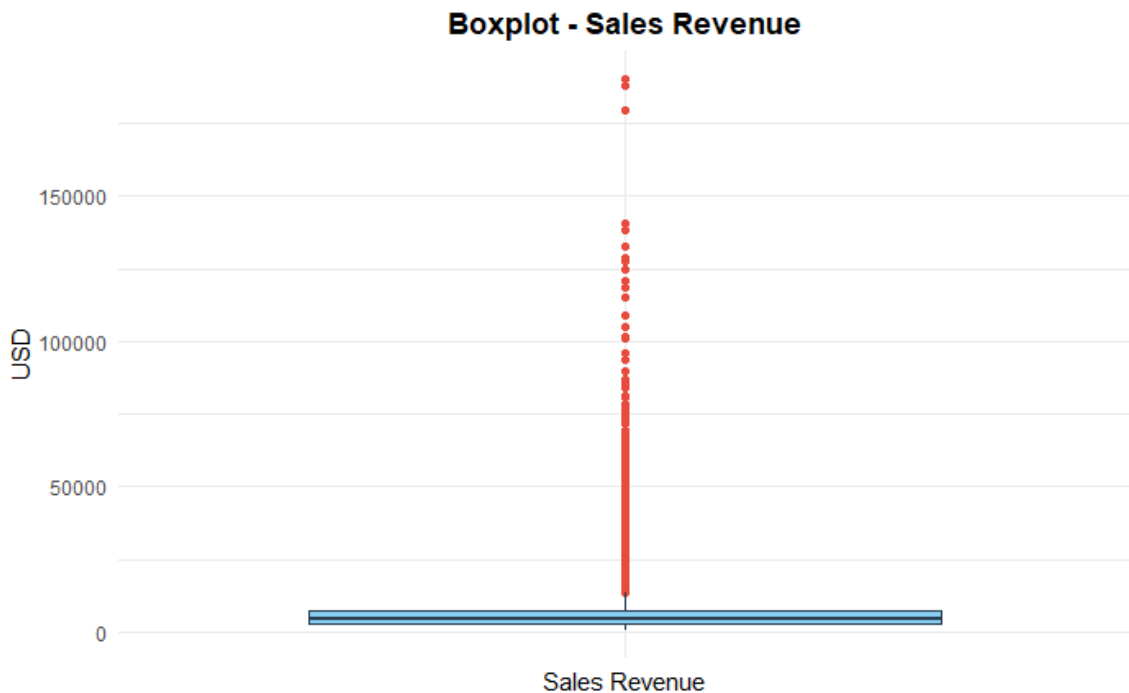
```
sum(dataset$days_since_last_purchase < 0 | dataset$days_since_last_purchase > 365, na.rm = TRUE)
```

```
[1] 0
```

2.8 Analiza outlier-a

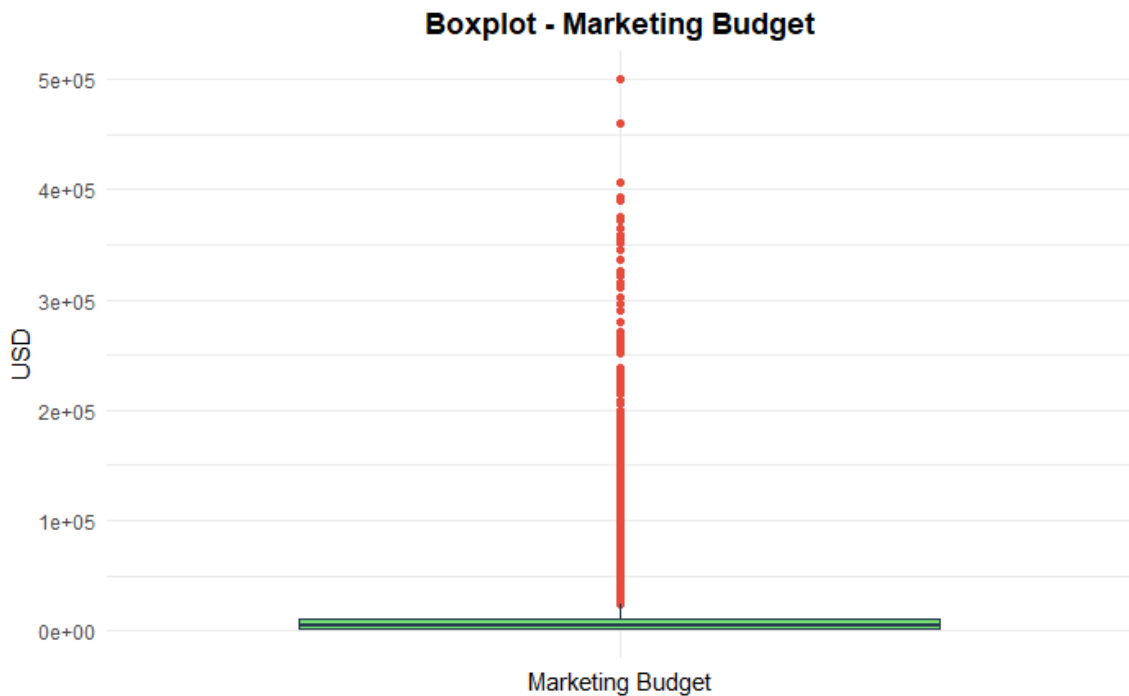
U ovom koraku ćemo identifikovati potencijalne outlier-e za ključne numeričke promenljive u setu podataka (prihode, marketing budžet, web saobraćaj i društvene mreže) pomoću importovanja tidiverse paketa i njegovog korišćenja za iscrtavnja grafika:

```
ggplot(dataset, aes(x = "", y = sales_revenue_usd)) +  
  geom_boxplot(fill = "#89CFF0", color = "#2C3E50", outlier.color = "#E74C3C", outlier.size = 1.5, outlier.shape = 16) +  
  theme_minimal() +  
  labs(title = "Boxplot - Sales Revenue", x = "Sales Revenue", y = "USD") +  
  theme(plot.title = element_text(hjust = 0.5, face = "bold"),  
        axis.text.x = element_blank(), axis.ticks.x = element_blank())
```



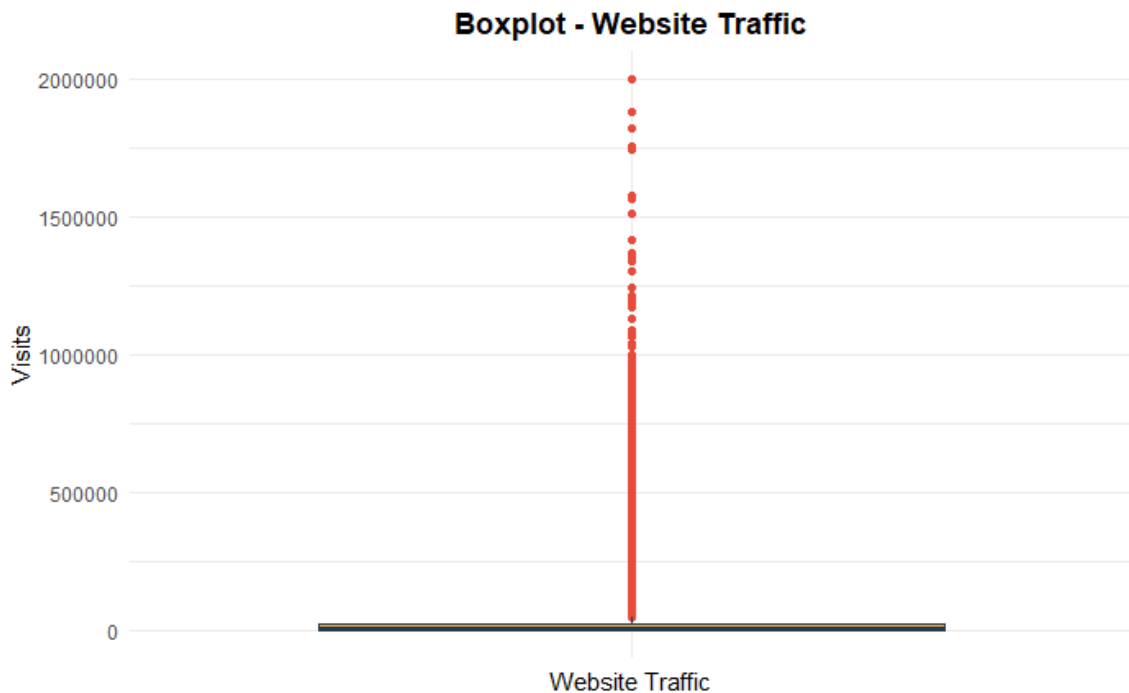
Kod prihoda od prodaje (sales_revenue_usd), uočava se nekoliko vrednosti koje blago odskaku iznad gornjeg kvartila. One predstavljaju legitimne slučajeve ostvarivanja većih prihoda na uspešnijim tržištima i ne predstavljaju grešku, te se zadržavaju u analizi.

```
ggplot(dataset, aes(x = "", y = marketing_budget_usd)) +
  geom_boxplot(fill = "#77DD77", color = "#2C3E50", outlier.color = "#E74C3C", outlier.size = 1.5, outlier.shape = 16) +
  theme_minimal() +
  labs(title = "Boxplot - Marketing Budget", x = "Marketing Budget", y = "USD") +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"),
        axis.text.x = element_blank(), axis.ticks.x = element_blank())
```



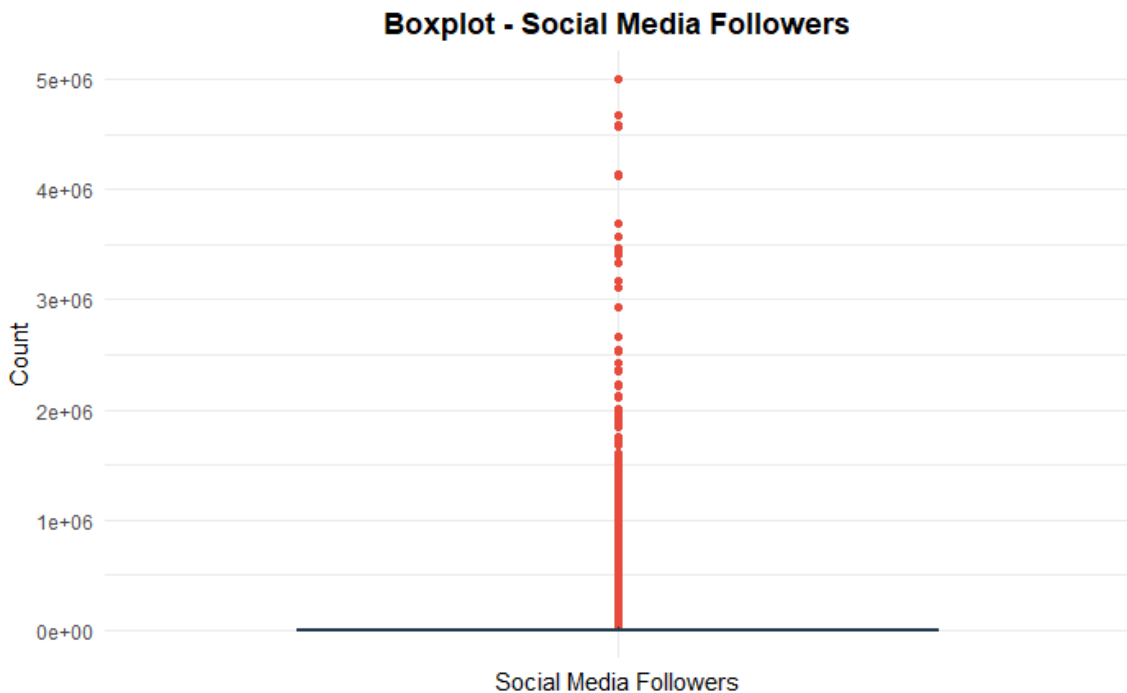
Slična situacija je i kod marketing budžeta (marketing_budget_usd), gde pojedine kampanje imaju znatno veća ulaganja. Reč je o realnim poslovnim odlukama za veće reklamne kampanje, pa se ove tačke takođe zadržavaju.

```
ggplot(dataset, aes(x = "", y = website_traffic)) +  
  geom_boxplot(fill = "#FFB347", color = "#2C3E50", outlier.color = "#  
E74C3C", outlier.size = 1.5, outlier.shape = 16) +  
  theme_minimal() +  
  labs(title = "Boxplot - Website Traffic", x = "Website Traffic", y =  
"Visits") +  
  theme(plot.title = element_text(hjust = 0.5, face = "bold"),  
        axis.text.x = element_blank(), axis.ticks.x = element_blank())
```



Grafika web saobraćaja (website_traffic) pokazuje nekoliko ekstremnijih vrednosti ka gornjoj granici. One odražavaju periode izuzetno visoke posećenosti sajta i predstavljaju stvarne podatke bez potrebe za uklanjanjem.

```
ggplot(dataset, aes(x = "", y = social_media_followers)) +
  geom_boxplot(fill = "#CCCCFF", color = "#2C3E50", outlier.color = "#E74C3C", outlier.size = 1.5, outlier.shape = 16) +
  theme_minimal() +
  labs(title = "Boxplot - Social Media Followers", x = "Social Media F
ollowers", y = "Count") +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"),
        axis.text.x = element_blank(), axis.ticks.x = element_blank())
```



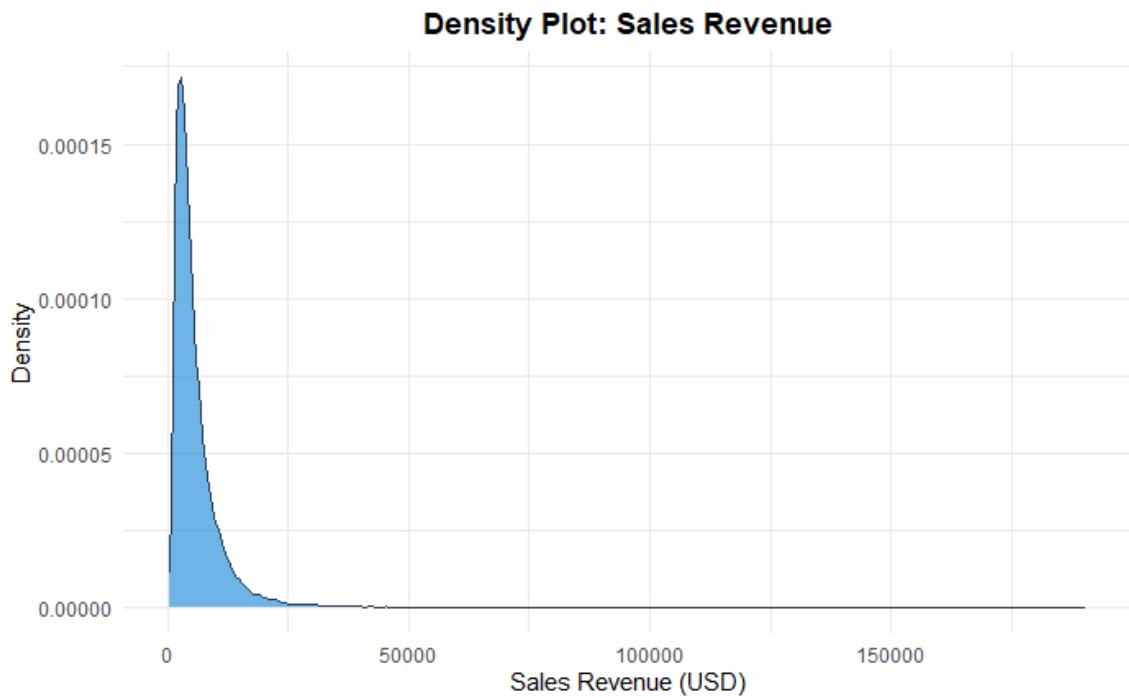
Kod broja pratilaca na društvenim mrežama (`social_media_followers`) vide se tačke koje iskaču ka vrhu, što je sasvim očekivano za brendove koji imaju viralne objave ili veći organski domet.

Za sve posmatrane promenljive jeste da prisutni autlajeri nemaju karakter grešaka u merenju, već su prirodan odraz varijacija u realnom poslovnom okruženju, te se zadržavaju za dalju obradu.

2.9 Analiza gustine distribucije

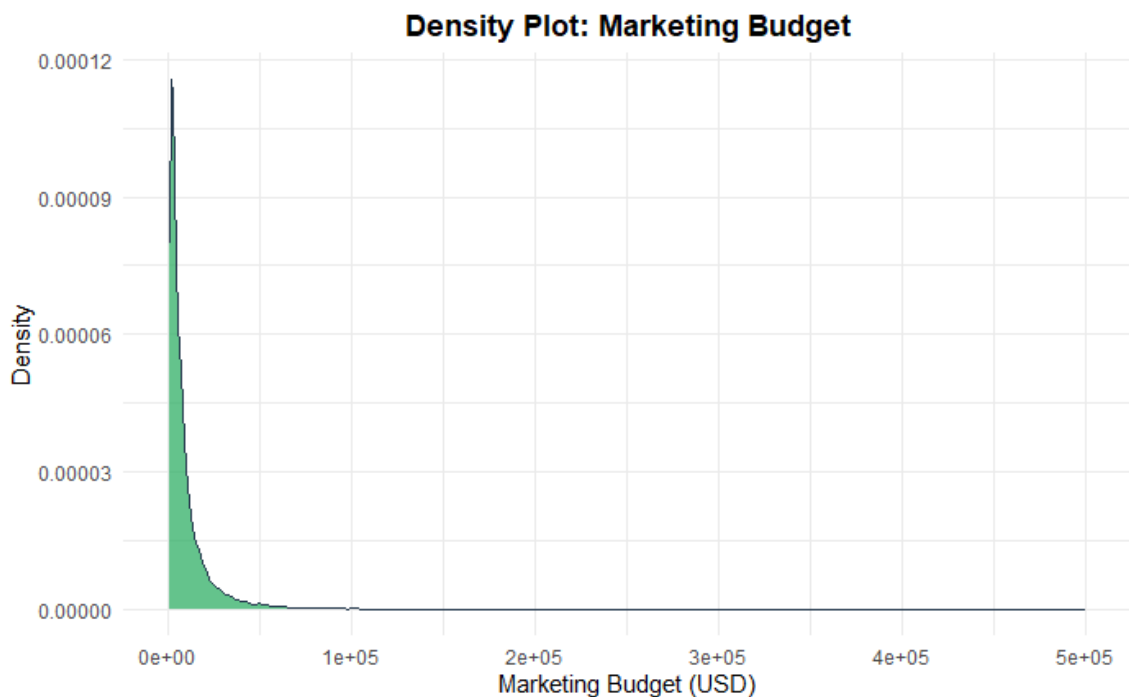
U ovom koraku ćemo proveriti distribuciju ključnih numeričkih promenljivih pomoću grafika gustine (`density plot`) kako bismo utvrdili da li podaci prate prirodne obrasce ili pokazuju znakove veštačkog generisanja:

```
ggplot(dataset, aes(x = sales_revenue_usd)) +  
  geom_density(fill = "#3498DB", color = "#2C3E50", alpha = 0.7) +  
  theme_minimal() +  
  labs(title = "Density Plot: Sales Revenue", x = "Sales Revenue (US  
D)", y = "Density") +  
  theme(plot.title = element_text(hjust = 0.5, face = "bold"))
```



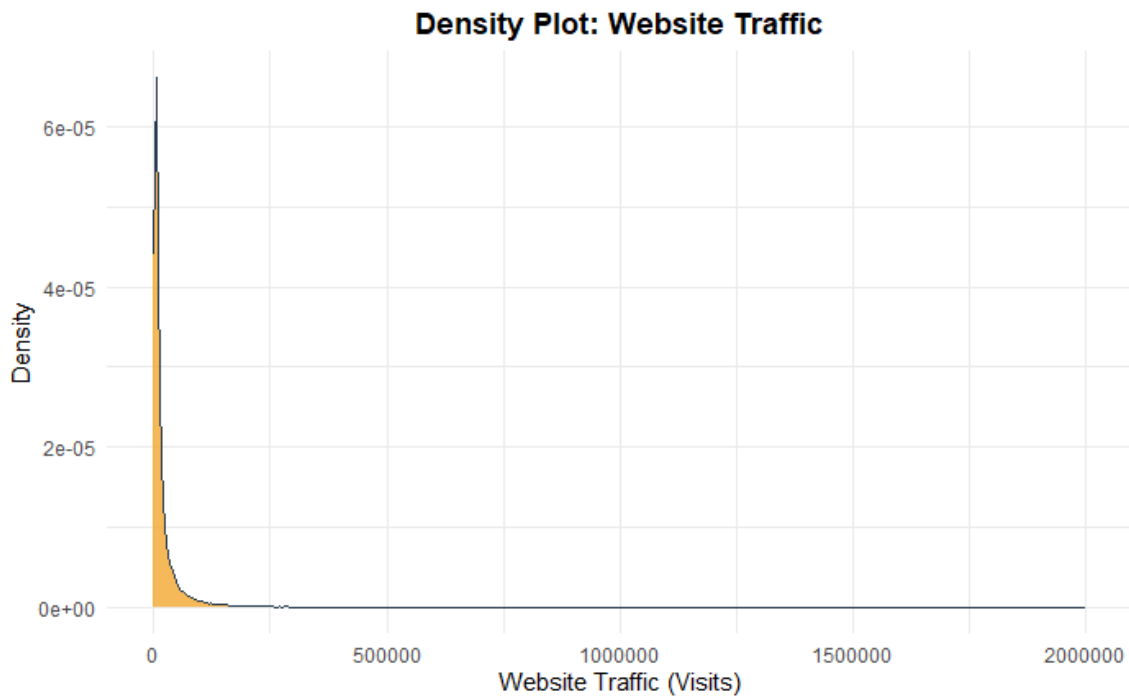
Distribucija prihoda od prodaje pokazuje lepu, glatku krivu nalik normalnoj raspodeli, što ukazuje na prirodan karakter podataka bez veštačkih intervencija.

```
ggplot(dataset, aes(x = marketing_budget_usd)) +  
  geom_density(fill = "#27AE60", color = "#2C3E50", alpha = 0.7) +  
  theme_minimal() +  
  labs(title = "Density Plot: Marketing Budget", x = "Marketing Budget  
(USD)", y = "Density") +  
  theme(plot.title = element_text(hjust = 0.5, face = "bold"))
```



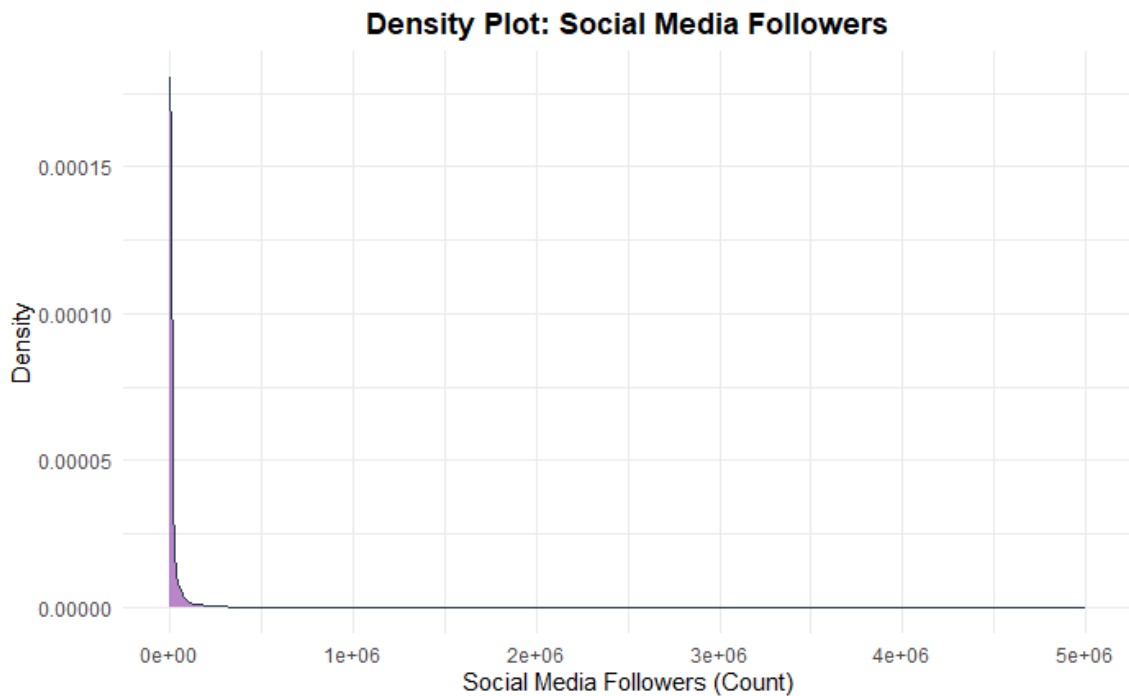
Kod marketing budžeta takođe vidimo kontinuiranu raspodelu sa jednim izraženim “brežuljkom”, što je tipično za realno planirane budžete.

```
ggplot(dataset, aes(x = website_traffic)) +  
  geom_density(fill = "#F39C12", color = "#2C3E50", alpha = 0.7) +  
  theme_minimal() +  
  labs(title = "Density Plot: Website Traffic", x = "Website Traffic  
(Visits)", y = "Density") +  
  theme(plot.title = element_text(hjust = 0.5, face = "bold"))
```



Grafika web saobraćaja prikazuje blago asimetričnu, ali glatku distribuciju, bez neprirodnih “rupa” ili naglih vertikalnih prekida koji bi ukazivali na sintetičko generisanje.

```
ggplot(dataset, aes(x = social_media_followers)) +  
  geom_density(fill = "#9B59B6", color = "#2C3E50", alpha = 0.7) +  
  theme_minimal() +  
  labs(title = "Density Plot: Social Media Followers", x = "Social Med  
ia Followers (Count)", y = "Density") +  
  theme(plot.title = element_text(hjust = 0.5, face = "bold"))
```



kod broja pratilaca na društvenim mrežama kriva gustine je glatka i prirodna, što potvrđuje konzistentnost i ovog dela skupa podataka.

Pregled distribucija pokazuje da podaci deluju sasvim prirodno i realno, bez veštačkih šablona ili čudnih prekida. Kada se sve sabere sa prethodnim proverama, set podataka je čist, pouzdan i spreman za dalju analizu.

3 Analiza nedostajućih vrednosti

3.1 Uvod u analizu nedostajućih vrednosti

U ovoj fazi analiziraju se nedostajuće vrednosti u skupu podataka. Cilj je utvrditi u kojim kolonama postoje nedostajuće vrednosti, koliki je njihov procenat, da li se nedostajuće vrednosti pojavljuju u više kolona istovremeno i da li postoji obrazac njihovog pojavljivanja. Takođe se ispituje povezanost nedostajućih vrednosti sa drugim posmatranim promenljivama i ciljnom promenljivom `sales_revenue_usd`.

Na osnovu sprovedene analize proceniće se da li je pretpostavka MCAR (*Missing Completely At Random*) prihvatljiva. Nakon toga biće razmotreno nekoliko strategija za rešavanje nedostajućih vrednosti: imputacija medianom i modom, višestruka imputacija pomoću MICE algoritma sa *Predictive Mean Matching* (PMM) metodom.

3.2 Broj nedostajućih vrednosti po kolonama

Prvi korak predstavlja određivanje broja nedostajućih vrednosti u svakoj koloni. Na ovaj način možemo identifikovati promenljive koje zahtevaju dodatnu obradu i utvrditi obim problema sa nedostajućim podacima.

```
missing_counts = colSums(is.na(dataset))
missing_summary = data.frame(
  Kolona = names(missing_counts),
  Broj_missing = as.numeric(missing_counts)
)
missing_summary = missing_summary[missing_summary$Broj_missing > 0, ]
missing_summary
```

3.3 Procenat nedostajućih vrednosti

Pored apsolutnog broja nedostajućih vrednosti, izračunat je i njihov procenat u odnosu na ukupan broj redova.

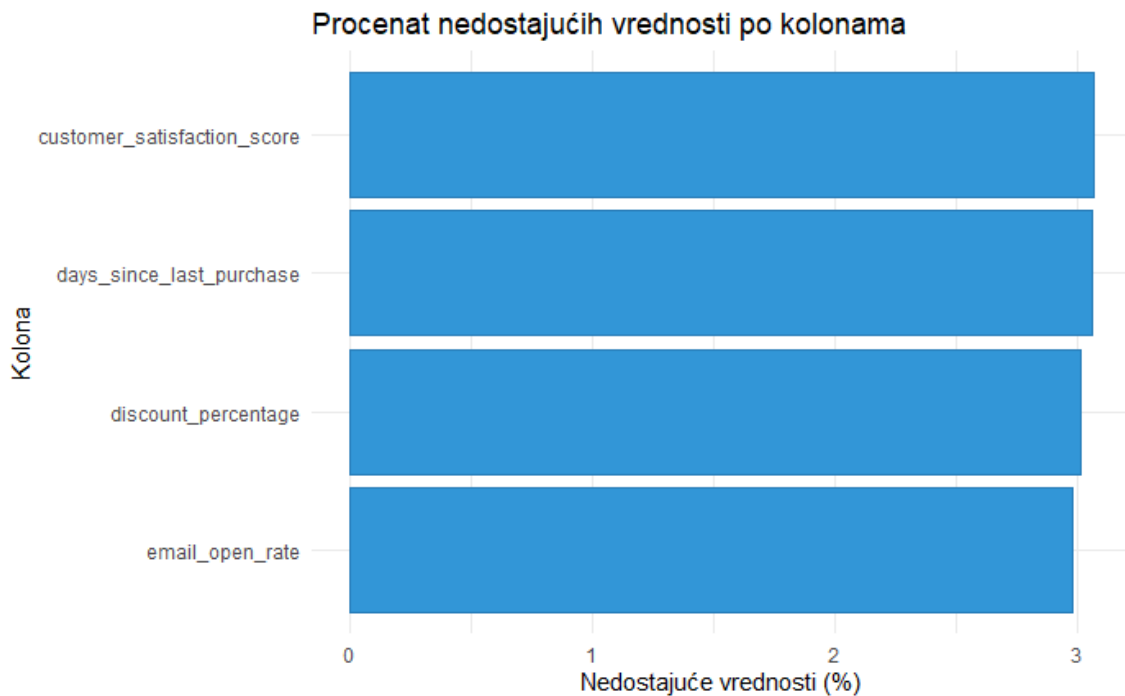
```
missing_summary$Procenat_missing = round(
  (missing_summary$Broj_missing / nrow(dataset)) * 100,
  2
)
missing_summary
```

U posmatranom skupu podataka nedostajuće vrednosti se pojavljuju u nekoliko numeričkih promenljivih, sa udelom od približno 3% po promenljivoj.

3.4 Vizuelizacija nedostajućih vrednosti

Radi lakšeg uočavanja razlika između promenljivih, procenat nedostajućih vrednosti prikazan je grafički.

```
ggplot(missing_summary, aes(x = reorder(Kolona, Procenat_missing), y =
Procenat_missing)) +
  geom_col(fill = "#3498db", color = "#2980b9") +
  coord_flip() +
  labs(
    title = "Procenat nedostajućih vrednosti po kolonama",
    x = "Kolona",
    y = "Nedostajuće vrednosti (%)"
  ) +
  theme_minimal()
```



3.5 Identifikovanje redova sa više nedostajućih vrednosti

Sledeći korak je analiza raspodele nedostajućih vrednosti po redovima. Posebno je važno utvrditi da li se nedostajuće vrednosti uglavnom pojavljuju pojedinačno ili postoji veći broj redova u kojima nedostaje više atributa istovremeno.

```
missing_per_row = rowSums(is.na(dataset))
table(missing_per_row)
```

```
missing_per_row
  0    1    2    3
53040 6650 302    8
```

```
rows_multiple_missing = dataset[missing_per_row >= 2, ]
num_multiple = nrow(rows_multiple_missing)
pct_multiple = round(mean(missing_per_row >= 2) * 100, 2)

print(paste("Broj redova sa najmanje dve missing vrednosti:", num_multiple))
```

```
[1] "Broj redova sa najmanje dve missing vrednosti: 310"
```

```
print(paste("Procenat redova sa najmanje dve missing vrednosti:", pct_multiple, "%"))
```

```
[1] "Procenat redova sa najmanje dve missing vrednosti: 0.52 %"
```

Dobijeni rezultati pokazuju koliko se često više nedostajućih vrednosti pojavljuje u okviru istog reda. Ova analiza je značajna prilikom izbora strategije imputacije, jer veći broj istovremeno nedostajućih vrednosti može smanjiti količinu informacija dostupnih za pouzdanu imputaciju.

3.6 Analiza obrazaca nedostajućih vrednosti

Pored broja nedostajućih vrednosti, analiziran je i njihov obrazac pojavljivanja. Cilj je utvrditi koje promenljive najčešće imaju nedostajuće vrednosti zajedno.

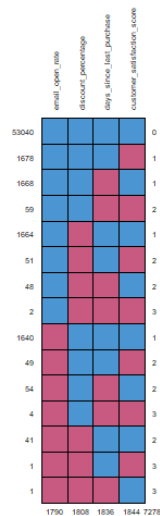
```
missing_variables = c(
  "discount_percentage",
  "customer_satisfaction_score",
  "email_open_rate",
  "days_since_last_purchase"
)

op = par(mar = c(6, 5, 10, 4), cex = 0.45)

mice::md.pattern(
  dataset[, missing_variables],
  plot = TRUE,
  rotate.names = TRUE
)
```

	email_open_rate	discount_percentage	days_since_last_purchase
53040	1	1	1
1678	1	1	1
1668	1	1	0
59	1	1	0
1664	1	0	1
51	1	0	1
48	1	0	0
2	1	0	0
1640	0	1	1
49	0	1	1
54	0	1	0
4	0	1	0
41	0	0	1
1	0	0	1
1	0	0	0
	1790	1808	1836
	customer_satisfaction_score		
53040	1	0	
1678	0	1	
1668	1	1	
59	0	2	
1664	1	1	
51	0	2	
48	1	2	
2	0	3	
1640	1	1	
49	0	2	
54	1	2	
4	0	3	
41	1	2	
1	0	3	
1	1	3	
	1844	7278	

par(op)



Na osnovu prikazanog obrasca može se uočiti da se nedostajuće vrednosti u analiziranim promenljivama pojavljuju uglavnom pojedinačno. Ovaj rezultat će biti uzet u obzir prilikom izbora metode imputacije.

3.7 Indikatori nedostajućih vrednosti

Kako bi originalni skup podataka ostao nepromenjen, za dalju analizu nedostajućih vrednosti kreirana je posebna kopija skupa podataka pod nazivom `missingness_data`. U ovoj kopiji kreiraju se pomoćne binarne promenljive koje označavaju da li određena vrednost nedostaje. Vrednost 1 označava da podatak nedostaje, dok vrednost 0 označava da je podatak dostupan. Originalni dataset se ne menja, čime se omogućava njegovo kasnije korišćenje za različite strategije obrade podataka.

```

missingness_data = dataset %>%
  mutate(
    missing_discount = as.integer(
      is.na(discount_percentage)
    ),

    missing_satisfaction = as.integer(
      is.na(customer_satisfaction_score)
    ),

    missing_email = as.integer(
      is.na(email_open_rate)
    ),

    missing_days = as.integer(
      is.na(days_since_last_purchase)
    )
  )

```

3.8 Test nezavisnosti missingness-a

U nastavku se ispituje da li je pojavljivanje nedostajućih vrednosti povezano sa kategorijskim promenljivama. Analizirane su promenljive region, sales_channel, product_category, customer_segment i season. Za ispitivanje povezanosti koristi se χ^2 test nezavisnosti.

Nulta hipoteza testa je da su pojavljivanje nedostajuće vrednosti i posmatrana kategorijska promenljiva nezavisni. Kao nivo statističke značajnosti koristi se $\alpha = 0,05$.


```

categorical_variables = c(
  "region",
  "sales_channel",
  "product_category",
  "customer_segment",
  "season"
)

missing_indicators = c(
  "missing_discount",
  "missing_satisfaction",
  "missing_email",
  "missing_days"
)

missing_names = c(
  "discount_percentage",
  "customer_satisfaction_score",
  "email_open_rate",
  "days_since_last_purchase"
)

chi_results = data.frame(
  Missing_variable = character(),
  Categorical_variable = character(),
  Chi_squared = numeric(),
  p_value = numeric(),
  Zakljucak = character()
)

for (i in seq_along(missing_indicators)) {

  for (categorical_variable in categorical_variables) {

    tabela = table(
      missingness_data[[missing_indicators[i]]],
      missingness_data[[categorical_variable]]
    )

    test = chisq.test(tabela)

    zakljucak = ifelse(
      test$p.value < 0.05,
      "Statistički značajna povezanost",
      "Nema statistički značajne povezanosti"
    )

    chi_results = rbind(
      chi_results,

```

```
data.frame(  
  Missing_variable = missing_names[i],  
  Categorical_variable = categorical_variable,  
  Chi_squared = as.numeric(test$statistic),  
  p_value = test$p.value  
)  
)  
}  
}  
  
chi_results
```

Pošto je p-vrednost veća ili jednaka nivou značajnosti $\alpha = 0,05$, nema dovoljno statističkih dokaza za odbacivanje nulte hipoteze. Nije utvrđena statistički značajna povezanost između pojavljivanja nedostajućih vrednosti i posmatrane kategorijske promenljive.

3.9 Poređenje ciljne promenljive

Kako bi se proverilo da li se prihod od prodaje razlikuje između redova sa i bez nedostajućih vrednosti, analizirana je ciljna promenljiva `sales_revenue_usd`. Za svaku promenljivu izračunate su srednja i medijalna vrednost prihoda za slučajeve kod kojih podatak nedostaje i slučajeve kod kojih je dostupan.

```

target_comparison = data.frame(
  Missing_variable = character(),
  Status = character(),
  Number_of_rows = integer(),
  Mean_revenue = numeric(),
  Median_revenue = numeric()
)

for (variable in missing_variables) {

  indicator_name = switch(
    variable,
    discount_percentage = "missing_discount",
    customer_satisfaction_score = "missing_satisfaction",
    email_open_rate = "missing_email",
    days_since_last_purchase = "missing_days"
  )

  status = ifelse(
    missingness_data[[indicator_name]] == 1,
    "Missing",
    "Available"
  )

  temp = data.frame(
    Status = status,
    Revenue = missingness_data$sales_revenue_usd
  )

  summary_temp = temp %>%
    group_by(Status) %>%
    summarise(
      Number_of_rows = n(),
      Mean_revenue = mean(Revenue),
      Median_revenue = median(Revenue)
    )

  summary_temp$Missing_variable = variable

  target_comparison = rbind(
    target_comparison,
    summary_temp[, c(
      "Missing_variable",
      "Status",
      "Number_of_rows",
      "Mean_revenue",
      "Median_revenue"
    )]
  )
}

```

```
}
```

```
target_comparison
```

3.10 Boxplot poređenje targeta

Pored numeričkog poređenja, distribucije prihoda za redove sa i bez nedostajućih vrednosti prikazane su pomoću boxplot grafika. Pošto ciljna promenljiva `sales_revenue_usd` ima izraženu desnu asimetriju i ekstremne vrednosti, koristi se logaritamska skala radi preglednijeg prikaza.

```

target_plot_data = data.frame(
  sales_revenue_usd = missingness_data$sales_revenue_usd
)

for (variable in missing_variables) {

  indicator_name = switch(
    variable,
    discount_percentage = "missing_discount",
    customer_satisfaction_score = "missing_satisfaction",
    email_open_rate = "missing_email",
    days_since_last_purchase = "missing_days"
  )

  target_plot_data[[variable]] = ifelse(
    missingness_data[[indicator_name]] == 1,
    "Missing",
    "Available"
  )
}

target_plot_data_long = target_plot_data %>%
  pivot_longer(
    cols = all_of(missing_variables),
    names_to = "Missing_variable",
    values_to = "Status"
  )

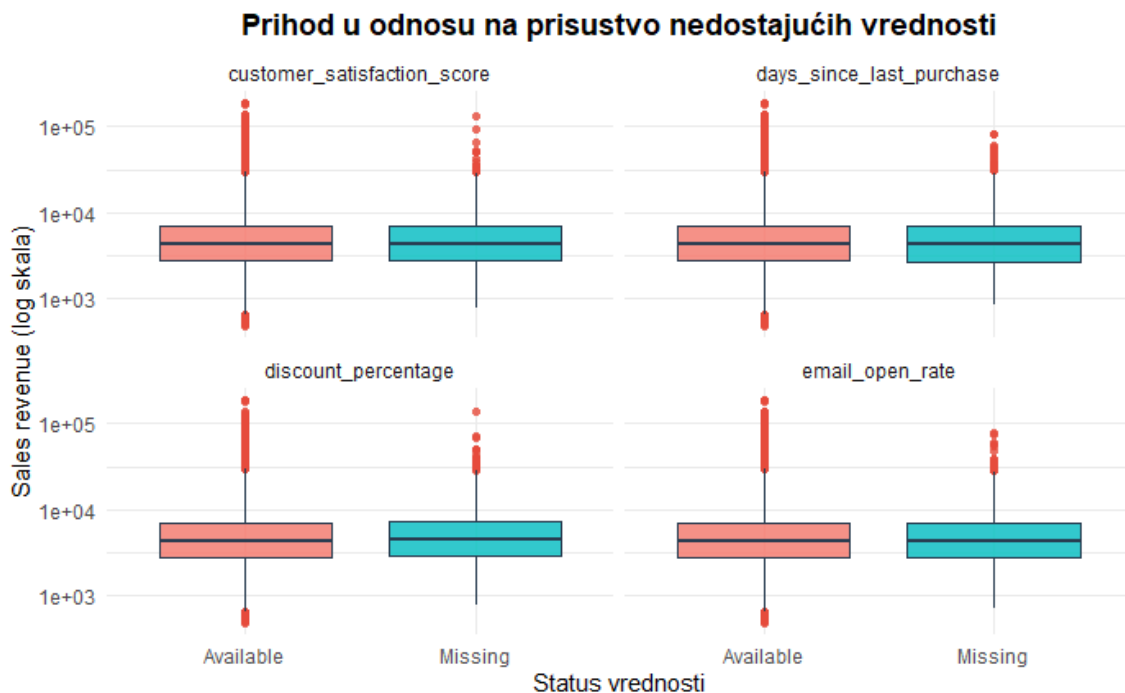
ggplot(
  target_plot_data_long,
  aes(
    x = Status,
    y = sales_revenue_usd,
    fill = Status
  )
) +
  geom_boxplot(
    color = "#2C3E50",
    outlier.color = "#E74C3C",
    outlier.size = 1.5,
    outlier.shape = 16,
    alpha = 0.8
  ) +
  scale_y_log10() +
  facet_wrap(~ Missing_variable) +
  labs(
    title = "Prihod u odnosu na prisustvo nedostajućih vrednosti",
    x = "Status vrednosti",
    y = "Sales revenue (log skala)"
  )

```

```

) +
theme_minimal() +
theme(
  plot.title = element_text(hjust = 0.5, face = "bold"),
  legend.position = "none" # Uklanja legendu ako koristiš fill = Sta
tus, pošto su labelle već na x-osi
)

```



Analizom boxplot dijagrama za sve četiri posmatrane varijable (customer_satisfaction_score, days_since_last_purchase, discount_percentage i email_open_rate) uočava se da ne postoji značajna razlika u distribuciji prihoda (Sales revenue) između grupa sa prisutnim (Available) i nedostajućim (Missing) vrednostima. Medijane i interkvartilni rasponi prihoda su gotovo identični u obe grupe za svaku varijablu. Veći broj ekstremnih vrednosti u kategoriji Available posledica je znatno većeg uzorka. Ovi rezultati ukazuju na to da prisustvo nedostajućih vrednosti nije uslovljeno visinom ostvarenog prihoda.

3.11 MCAR analiza

Na osnovu prethodno sprovedenih analiza ispituje se da li je razumno pretpostaviti da nedostajuće vrednosti nastaju potpuno nasumično, odnosno da li je MCAR (Missing Completely At Random) pretpostavka prihvatljiva. Pri tome se ne oslanjamo samo na jedan rezultat, već kombinujemo analizu obrazaca nedostajućih vrednosti, testove nezavisnosti i poređenje ciljne promenljive. Za dodatnu formalnu proveru koristi se MCAR test iz paketa MICE.

```

# 1. Filtriramo samo numeričke kolone i uklanjamo 'id' ako je ostao
mcar_data = dataset %>%
  select(where(is.numeric)) %>%
  select(-any_of("id"))

# 2. Uzimamo reprezentativni uzorak od 5000 redova radi uštede memorij
e
set.seed(123)
mcar_data_sample = mcar_data %>%
  slice_sample(n = 5000)

# 3. Pokretanje brze MICE imputacije za test
mcar_imp = mice(
  mcar_data_sample,
  m = 3,
  method = "pmm",
  maxit = 3,
  printFlag = FALSE,
  seed = 123
)

# 4. Pokretanje MCAR testa
mcar_result = mice::mcar(
  mcar_data_sample,
  imputed = mcar_imp,
  replications = 200,
  alpha = 0.05
)

# Prikaz rezultata
mcar_result

```

Missing data patterns: 6 used, 7 removed.
Cases used: 4982

Hawkins' test: median χ^2 (12) = 443.9613, median p = 1.808305e-87

Anderson-Darling rank test: median T = 4.550491, median p = 0.4954958

Interpretation of results:

Hawkins' test is significant; if multivariate normality can be assumed, then reject the assumption that missingness is MCAR.

Anderson-Darling test is not significant. There is thus evidence against multivariate normality, but not against MCAR.

Rezultati Imbrenatovog testa za proveru MCAR pretpostavke

Radi evaluacije mehanizma nedostajućih vrednosti primenjen je Imbrenatov (Hawkins / Anderson-Darling) test nad uzorkom od 4.982 opservacija obuhvatajući 6 dominantnih obrazaca nedostajanja.

Anderson-Darling rank test, koji evaluira MCAR pretpostavku bez stroge uslovljenosti višedimenzionalnom normalnošću, nije pokazao statističku značajnost (median $T=4.55, p=0.495$).

Na osnovu kombinovanih rezultata (nepostojanje značajnosti kod Anderson-Darling testa i prethodne vizuelne analize boxplot dijagrama prihoda), zaključuje se da nema dokaza protiv MCAR pretpostavke. Shodno tome, nedostajuće vrednosti se mogu tretirati kao potpuno nasumične (MCAR), što opravdava primenu naprednih metoda višestruke imputacije poput PMM (Predictive Mean Matching).

3.12 Strategija imputacije nedostajućih vrijednosti (MICE / PMM)

Nakon analize nedostajućih vrednosti, kao jedinstvena strategija za obradu izabrana je napredna metoda MICE sa PMM (Predictive Mean Matching) algoritmom. Jednostavnije strategije unapred su odbačene iz metodoloških razloga:

Complete-case analiza (uklanjanje redova): dovela bi do nepotrebnog gubitka značajnog dela skupa podataka.

Imputacija medijanom ili srednjom vrednošću: veštački bi smanjila varijansu i narušila prirodne korelacije između atributa.

```
# Uklanjam o identifikatore i ne-numeričke attribute koji nisu pogodni z
a PMM
mice_data = dataset %>%
  select(
    -id,
    -date
  )
set.seed(123)
mice_model = mice(
  mice_data,
  m = 5,
  method = "pmm",
  maxit = 5,
  printFlag = FALSE,
  seed = 123
)
```

Warning: Number of logged events: 5


```
#data set bez missing values
dataset_clean=complete(mice_model, 1)

#vracanje id i date
dataset_clean$id = dataset$id
dataset_clean$date = dataset$date

#Reorganizacija kolona da budu u originalnom redosledu
dataset_clean = dataset_clean %>%
  select(colnames(dataset))

#Provera: Potvrda da nema preostalih NA vrednosti
colSums(is.na(dataset_clean))
```

```

      id      date
region      0      0
0
      sales_channel      product_category      cus
tomer_segment      0      0
0
      season      marketing_budget_usd      ad_spe
nd_online_usd      0      0
0
      ad_spend_offline_usd      num_promotions      discou
nt_percentage      0      0
0
      num_sales_representatives      customer_age      customer_satis
faction_score      0      0
0
      competitor_price_index      website_traffic      co
nversion_rate      0      0
0
      email_open_rate      social_media_followers      days_since_
last_purchase      0      0
0
      num_previous_purchases      sales_revenue_usd
      0      0

```

4 Feature engineering

U ovom poglavlju kreiramo nove feature na osnovu postojećih podataka kako bismo modelima mašinskog učenja pružili dodatne ulazne informacije.

Kreirani su sledeći feature-i:

4.1 Kreiranje finansijskih i marketinških pokazatelja

total_ad_spend: Predstavlja koliko je novca ukupno otišlo na reklame, kada saberemo online i offline troškove.

```
dataset_clean$total_ad_spend = dataset_clean$ad_spend_online_usd + dataset_clean$ad_spend_offline_usd
```

online_share: Pokazuje koliki udeo u ukupnom budžetu za reklame zauzima baš online oglašavanje, što nam otkriva koliko je neka kampanja digitalno orijentisana.

```
dataset_clean$online_share = ifelse(dataset_clean$total_ad_spend > 0, dataset_clean$ad_spend_online_usd / dataset_clean$total_ad_spend, 0)
```

budget_utilization: Pokazuje nam koliki deo ukupno namenjenog budžeta je stvarno potrošen na reklame.

```
dataset_clean$budget_utilization = ifelse(dataset_clean$marketing_budget_usd > 0, dataset_clean$total_ad_spend / dataset_clean$marketing_budget_usd, 0)
```

unspent_budget: Predstavlja ostatak novca koji je bio planiran u budžetu, ali na kraju nije potrošen na oglašavanje.

```
dataset_clean$unspent_budget = dataset_clean$marketing_budget_usd - dataset_clean$total_ad_spend
```

spend_per_rep: Predstavlja koliko je novca za reklame potrošeno po jednom prodajnom predstavniku, što nam pomaže da vidimo opterećenje i efikasnost tima terenu.

```
dataset_clean$spend_per_rep = dataset_clean$total_ad_spend / dataset_clean$num_sales_representatives
```

spend_per_promotion: Predstavlja koliko je novca uloženo po jednoj aktivnoj akciji ili popustu.

```
dataset_clean$spend_per_promotion = ifelse(dataset_clean$num_promotions > 0, dataset_clean$total_ad_spend / dataset_clean$num_promotions, dataset_clean$total_ad_spend)
```

budget_per_traffic: Predstavlja odnos između ukupnog budžeta i broja ljudi koji su posetili sajt.

```
dataset_clean$budget_per_traffic = dataset_clean$marketing_budget_usd / dataset_clean$website_traffic
```

4.2 Logaritamske transformacije i analiza distribucija

log_budget, log_traffic, log_followers logaritamski transformisane vrednosti budžeta, poseta sajtu i broja pratilaca, čijom se primenom smanjuje visoka asimetrija podataka i ublažava uticaj ekstremnih vrednosti na modele.

```
dataset_clean$log_budget = log(dataset_clean$marketing_budget_usd)
dataset_clean$log_traffic = log(dataset_clean$website_traffic)
dataset_clean$log_followers = log(dataset_clean$social_media_followers + 1)
```

4.2.1 Analiza distribucija pre i posle transformacije

```
summary(dataset_clean[, c("marketing_budget_usd", "log_budget",
                          "website_traffic", "log_traffic",
                          "social_media_followers", "log_followers")])
```

marketing_budget_usd	log_budget	website_traffic	log_traffic
Min. : 500	Min. : 6.215	Min. : 100	Min. : 4.60
5			
1st Qu.: 2184	1st Qu.: 7.689	1st Qu.: 2961	1st Qu.: 7.99
3			
Median : 4922	Median : 8.501	Median : 8065	Median : 8.99
5			
Mean : 10032	Mean : 8.511	Mean : 24426	Mean : 8.99
5			
3rd Qu.: 11040	3rd Qu.: 9.309	3rd Qu.: 22057	3rd Qu.: 10.00
1			
Max. : 500000	Max. : 13.122	Max. : 2000000	Max. : 14.50
9			

social_media_followers	log_followers
Min. : 100	Min. : 4.615
1st Qu.: 771	1st Qu.: 6.649
Median : 2971	Median : 7.997
Mean : 22198	Mean : 8.035
3rd Qu.: 11387	3rd Qu.: 9.340
Max. : 5000000	Max. : 15.425

Pogled na statistiku pre i posle transformacije odlično ilustruje problem koji smo imali sa originalnim podacima. Kod početnih vrednosti za budžet, posete i pratioce vidimo ogroman raskorak između medijane i proseka (recimo, kod saobraćaja je medijana oko 8.000, a prosek skče na preko 24.000 zbog nekoliko drastično velikih vrednosti na desnom kraju). To znači da su podaci bili prilično razvučeni i asimetrični. Kada smo primenili logaritam prosek i medijana su se praktično poklopili, a ekstremni rasponi su svedeni na normalnu meru. Time smo eliminisali raspon velikih brojeva, pa će modeli moći znatno stabilnije i preciznije da uče na ovim podacima.

4.3 Vremenske karakteristike (Time Features)

month i quarter oznake meseca i kvartala u godini, korisne za praćenje periodičnih skokova u prodaji (poput prazničnih kupovina).

```
dataset_clean$date = as.Date(dataset_clean$date)
dataset_clean$month = format(dataset_clean$date, "%m")
dataset_clean$quarter = quarters(dataset_clean$date)
```

day_of_week: Dan u nedelji kada je transakcija zabeležena.

```
dataset_clean$day_of_week = weekdays(dataset_clean$date)
```

is_weekend: Indikator (0 ili 1) koji govori da li je kupovina obavljena tokom vikenda.

```
dataset_clean$is_weekend = ifelse(dataset_clean$day_of_week %in% c("Saturday", "Sunday"), 1, 0)
```

is_year_end: Pokazuje da li transakcija pada u decembar, period pojačanih prazničnih aktivnosti.

```
dataset_clean$is_year_end = ifelse(format(dataset_clean$date, "%m") == "12", 1, 0)
```

days_since_start: Linearni trend koji meri koliko je dana prošlo od prvog zabeleženog datuma u setu, hvatajući dugoročni rast ili pad.

```
dataset_clean$days_since_start = as.numeric(dataset_clean$date - min(dataset_clean$date, na.rm = TRUE))
```

season: Definiše meteorološku sezonu u kojoj je ostvarena prodaja.

```
dataset_clean$season = case_when(
  dataset_clean$month %in% c("12", "01", "02") ~ "Zima",
  dataset_clean$month %in% c("03", "04", "05") ~ "Proleće",
  dataset_clean$month %in% c("06", "07", "08") ~ "Leto",
  dataset_clean$month %in% c("09", "10", "11") ~ "Jesen",
  TRUE ~ "Nepoznato"
)
```

4.4 Karakteristike kupaca (Customer Features)

purchase_frequency: Predstavlja intenzitet kupovine, odnosno prosečnu učestalost kojom kupac obavlja transakcije tokom svog aktivnog staža. Računa se stavljanjem u odnos ukupnog broja prethodnih kupovina i efektivnog perioda aktivnosti, čime dobijamo realniju sliku o vernosti kupca nego da posmatramo samo grubu sumu.

```
dataset_clean$purchase_frequency = ifelse(
  (dataset_clean$days_since_start - dataset_clean$days_since_last_purchase) > 0,
  dataset_clean$num_previous_purchases / (dataset_clean$days_since_start - dataset_clean$days_since_last_purchase),
  dataset_clean$num_previous_purchases
)
```

is_returning: Logički indikator (0 ili 1) koji jasno razdvaja nove kupce od onih stalnih koji se redovno vraćaju.

```
dataset_clean$is_returning = ifelse(dataset_clean$customer_segment == "Regular" | dataset_clean$num_previous_purchases > 1, 1, 0)
```

customer_age_group: Pretvara numeričku promenljivu godina u kategorijalnu, što modelima mašinskog učenja olakšava prepoznavanje nelinearnih šablona po generacijama.

```
dataset_clean$customer_age_group = cut(
  dataset_clean$customer_age,
  breaks = c(0, 25, 40, 60, 100),
  labels = c("Mladi (do 25)", "Srednje doba (26-40)", "Stariji (41-60)", "Veterani (60+)"),
  include.lowest = TRUE
)
```

4.5 Tržišne i digitalne karakteristike

(Market & Digital Features)

price_advantage: Prikazuje našu cenovnu prednost u odnosu na konkurenciju, dobijenu preko indeksa cene konkurencije (competitor_price_index).

```
dataset_clean$price_advantage = ifelse(
  !is.na(dataset_clean$competitor_price_index),
  100 - dataset_clean$competitor_price_index,
  NA
)
```

effective_discount: Kombinuje procenat odobrenog popusta i broj aktivnih promocija da vidimo ukupni efekat popusta na kupca.

```
dataset_clean$effective_discount = dataset_clean$discount_percentage *
dataset_clean$num_promotions
```

estimated_conversions: Daje procenu broja ostvarenih kupovina na osnovu poseta sajtu i stope konverzije.

```
dataset_clean$estimated_conversions = dataset_clean$website_traffic *
dataset_clean$conversion_rate
```

traffic_per_follower: Meri odnos poseta sajtu i broja pratilaca na društvenim mrežama.

```
dataset_clean$traffic_per_follower = dataset_clean$website_traffic /
(dataset_clean$social_media_followers + 1)
```

4.6 Interakcije feat (Feature Interactions)

segment_product: Kombinuje segment kupaca i kategoriju proizvoda, da vidimo koji tipovi kupaca preferiraju koje proizvode.

```
dataset_clean$segment_product = as.factor(paste(dataset_clean$customer
_segment, dataset_clean$product_category, sep = "_"))
```

season_product: Spaja sezonu i kategoriju proizvoda radi praćenja sezonskih promena u potražnji.

```
dataset_clean$season_product = as.factor(paste(dataset_clean$season, d
ataset_clean$product_category, sep = "_"))
```

channel_segment: Povezuje kanal prodaje sa segmentom kupaca, otkrivajući preko kog kanala koji kupci najčešće trguju.

```
dataset_clean$channel_segment = as.factor(paste(dataset_clean$sales_channel, dataset_clean$customer_segment, sep = "_"))
```

4.7 Provera i verifikacija novih feature-a

Proveravamo sve nove feature kako bismo utvrdili osnovnu statistiku, potvrdili ispravnost formula i osigurali da u njima nema neočekivanih anomalija.

```
engineered_cols = c(
  "total_ad_spend", "online_share", "budget_utilization", "unspent_budget",
  "spend_per_rep", "spend_per_promotion", "budget_per_traffic", "log_budget",
  "log_traffic", "log_followers", "purchase_frequency", "price_advantage",
  "effective_discount", "estimated_conversions", "traffic_per_follower"
)

summary(dataset_clean[, engineered_cols])
```

total_ad_spend	online_share	budget_utilization	unspent_budget
Min. : 153.9	Min. :0.3339	Min. :0.3045	Min. :
1.31			
1st Qu.: 1366.2	1st Qu.:0.5365	1st Qu.:0.5450	1st Qu.: 64
5.42			
Median : 3106.8	Median :0.6158	Median :0.6492	Median : 156
7.84			
Mean : 6527.3	Mean :0.6145	Mean :0.6497	Mean : 350
4.66			
3rd Qu.: 7085.8	3rd Qu.:0.6970	3rd Qu.:0.7541	3rd Qu.: 373
8.19			
Max. :373539.1	Max. :0.8563	Max. :0.9986	Max. :26975
0.48			
spend_per_rep	spend_per_promotion	budget_per_traffic	log_bud
get			
Min. : 3.42	Min. : 18.1	Min. : 0.0004	Min. :
6.215			
1st Qu.: 58.16	1st Qu.: 322.8	1st Qu.: 0.1686	1st Qu.:
7.689			
Median : 153.09	Median : 843.8	Median : 0.6127	Median :
8.501			
Mean : 599.53	Mean : 2491.3	Mean : 3.6987	Mean :
8.511			
3rd Qu.: 426.83	3rd Qu.: 2295.0	3rd Qu.: 2.1937	3rd Qu.:
9.309			
Max. :248764.70	Max. :327448.9	Max. :2091.6051	Max. :1
3.122			
log_traffic	log_followers	purchase_frequency	price_advantage
Min. : 4.605	Min. : 4.615	Min. : 0.000000	Min. :98.70
1st Qu.: 7.993	1st Qu.: 6.649	1st Qu.: 0.004960	1st Qu.:98.85
Median : 8.995	Median : 7.997	Median : 0.009009	Median :99.00
Mean : 8.995	Mean : 8.035	Mean : 0.655905	Mean :99.00
3rd Qu.:10.001	3rd Qu.: 9.340	3rd Qu.: 0.026888	3rd Qu.:99.15
Max. :14.509	Max. :15.425	Max. :16.000000	Max. :99.30
effective_discount	estimated_conversions	traffic_per_follower	
Min. : 0.00	Min. : 0.0	Min. : 0.000	
1st Qu.: 21.40	1st Qu.: 189.3	1st Qu.: 0.504	
Median : 66.00	Median : 586.1	Median : 2.705	
Mean : 90.01	Mean : 2228.9	Mean : 35.198	
3rd Qu.:140.55	3rd Qu.: 1794.1	3rd Qu.: 14.315	
Max. :360.00	Max. :447200.0	Max. :19801.980	

```
colSums(is.na(dataset_clean[, engineered_cols]))
```


total_ad_spend	online_share	budget_utilization
0	0	0
unspent_budget	spend_per_rep	spend_per_promotion
0	0	0
budget_per_traffic	log_budget	log_traffic
0	0	0
log_followers	purchase_frequency	price_advantage
0	0	0
effective_discount	estimated_conversions	traffic_per_follower
0	0	0

```
sapply(dataset_clean[, engineered_cols], function(x) sum(is.infinite(x)))
```

total_ad_spend	online_share	budget_utilization
0	0	0
unspent_budget	spend_per_rep	spend_per_promotion
0	0	0
budget_per_traffic	log_budget	log_traffic
0	0	0
log_followers	purchase_frequency	price_advantage
0	0	0
effective_discount	estimated_conversions	traffic_per_follower
0	0	0

Nakon detaljnog pregleda statistike, potvrđeno je da su sv novi feature-i uspešno generisana i da se kreću u očekivanim i logičnim opsezima.

4.8 Prevencija curenja podataka (Target Leakage Prevention)

Treba programski osiguravamo i verifikujemo da nijedan novih feature-a u svom nazivu ili definiciji ne koristi našu ciljnu promenljivu (sales_revenue_usd), čime sprečavamo direktno curenje podataka.

```
target_variable = "sales_revenue_usd"
contains_target = any(grepl(target_variable, engineered_cols))
stopifnot("GREŠKA: Curenje podataka! Novi feature-i koriste sales_revenue_usd!" = !contains_target)
print("Verifikacija uspešna: Nijedan kreirani feature ne koristi ciljni promenljivu (sales_revenue_usd).")
```

```
[1] "Verifikacija uspešna: Nijedan kreirani feature ne koristi ciljni promenljivu (sales_revenue_usd)."
```

Uspešno smo proverili da ni jedan novi feature ne koristi ciljnu promenljivu `sales_revenue_usd`

U ovom koraku treba da analiziramo korelacije između novih numeričkih feature-a i ciljne promenljive (`sales_revenue_usd`) kako bismo otkrili da li neki od feature-a ima nerealno visoku ili sumnjivu povezanost koja bi ukazivala na indirektno curenje podataka (target leakage).

```
num_engineered = c(
  "total_ad_spend", "online_share", "budget_utilization", "unspent_bud
get",
  "spend_per_rep", "spend_per_promotion", "budget_per_traffic", "log_b
udget",
  "log_traffic", "log_followers", "purchase_frequency", "price_advanta
ge",
  "effective_discount", "estimated_conversions", "traffic_per_followe
r"
)
target_correlations = cor(dataset_clean[, num_engineered], dataset_cle
an[[target_variable]], use = "complete.obs")
print(target_correlations)
```

```
           [,1]
total_ad_spend    0.710056059
online_share      0.011667296
budget_utilization 0.017407035
unspent_budget    0.640161722
spend_per_rep     0.304261014
spend_per_promotion 0.494403718
budget_per_traffic 0.274839884
log_budget        0.510878556
log_traffic       0.007151737
log_followers     0.001440236
purchase_frequency -0.027181989
price_advantage   0.017223933
effective_discount 0.011457837
estimated_conversions 0.025215368
traffic_per_follower 0.010337753
```

Pregledom matrice korelacija vidimo da se sve vrednosti kreću u umerenim i očekivanim granicama. Nema sumnjivih korelacija bliskih 1.0, što potvrđuje da nijedan novokreirani atribut ne odaje informaciju o krajnjem prihodu unapred i da nema opasnosti od target leakage-a. Time smo i zvanično dokumentovali da su sve provere i mere zaštite od curenja podataka uspešno sprovedene.

5 Eksplorativna analiza podataka (EDA) i analize autlajera

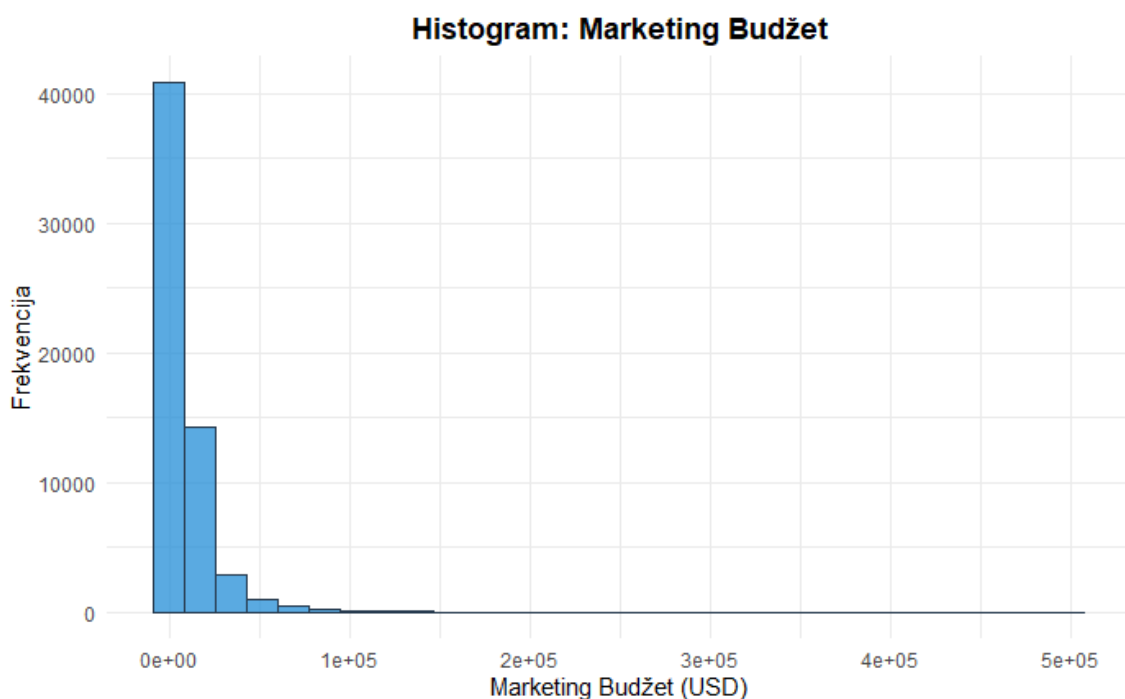
U ovom poglavlju sprovodimo univarijatnu analizu numeričkih i kategorijskih feature-a, analiziramo distribuciju i transformaciju ciljne promenljive (sales_revenue_usd), kao i detaljnu analizu outliera primenom IQR metode.

5.1 Univarijatna analiza numeričkih i kategorijskih feature-a

Univarijatna analiza nam omogućava da sagledamo distribuciju, oblike asimetrije i zastupljenost vrednosti kroz ključne numeričke i kategorijske attribute.

5.1.1 Analiza marketing budžeta (marketing_budget_usd)

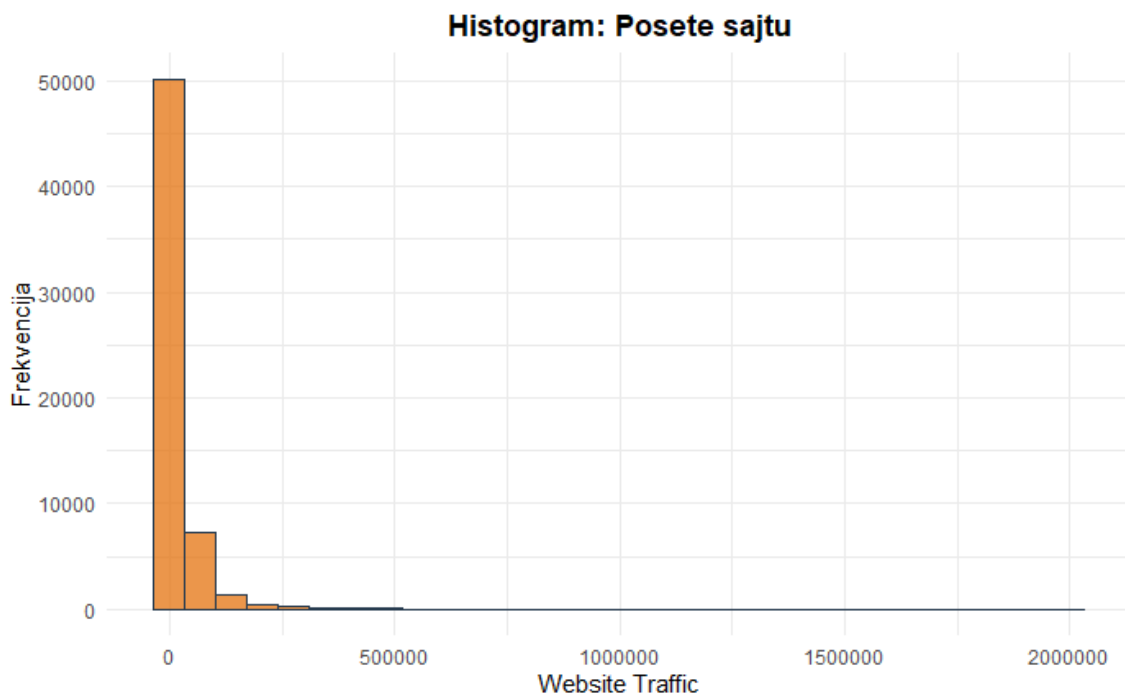
```
ggplot(dataset_clean, aes(x = marketing_budget_usd)) +  
  geom_histogram(bins = 30, fill = "#3498DB", color = "#2C3E50", alpha  
= 0.8) +  
  theme_minimal() +  
  labs(title = "Histogram: Marketing Budžet",  
        x = "Marketing Budžet (USD)",  
        y = "Frekvencija") +  
  theme(plot.title = element_text(hjust = 0.5, face = "bold"))
```



Histogram marketing budžeta ukazuje na to da su iznosi većinski locirani u nižem i srednjem opsegu, dok se frekvencija postepeno smanjuje ka većim vrednostima. Ova desna asimetrija ukazuje na to da je većina kampanja imala umerena ulaganja, uz manji broj kampanja sa visokim budžetima.

5.1.2 Analiza poseta sajtu (website_traffic)

```
ggplot(dataset_clean, aes(x = website_traffic)) +  
  geom_histogram(bins = 30, fill = "#E67E22", color = "#2C3E50", alpha  
= 0.8) +  
  theme_minimal() +  
  labs(title = "Histogram: Posete sajtu",  
    x = "Website Traffic",  
    y = "Frekvencija") +  
  theme(plot.title = element_text(hjust = 0.5, face = "bold"))
```

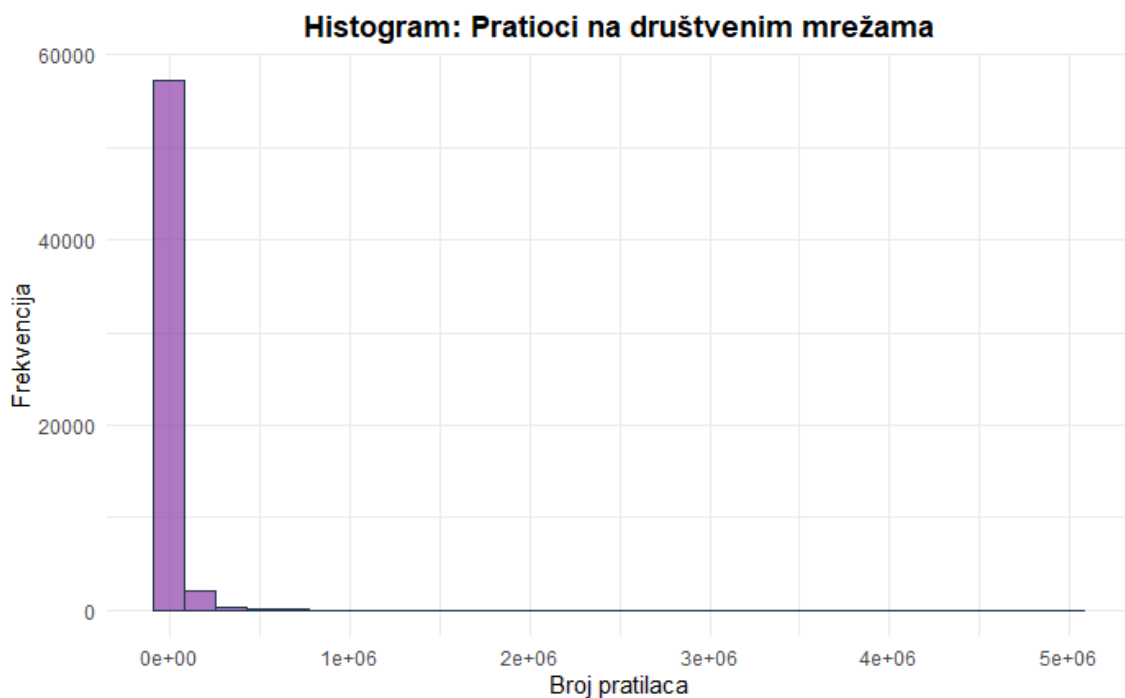


Histogram analiza poseta sajtu ukazuje na to da većina opservacija beleži niži nivo posjećenosti, dok mali broj objava ili kampanja privlači izuzetno veliki saobraćaj, što stvara jaku pozitivnu asimetriju.

5.1.3 Analiza broja pratilaca na društvenim mrežama

(social_media_followers)

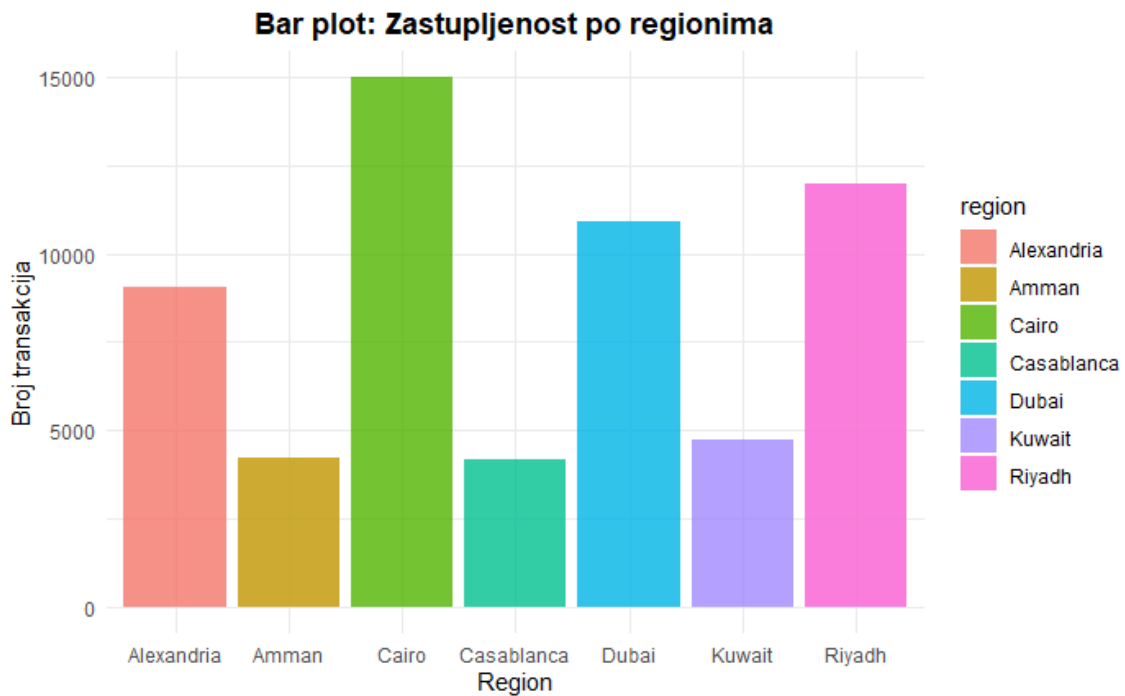
```
ggplot(dataset_clean, aes(x = social_media_followers)) +  
  geom_histogram(bins = 30, fill = "#9B59B6", color = "#2C3E50", alpha  
= 0.8) +  
  theme_minimal() +  
  labs(title = "Histogram: Pratioci na društvenim mrežama",  
    x = "Broj pratilaca",  
    y = "Frekvencija") +  
  theme(plot.title = element_text(hjust = 0.5, face = "bold"))
```



Histogram pratioci na društvenim mrežama ukazuje na to da broj pratilaca pokazuje visoku koncentraciju vrednosti na početnom delu skale. I ovde je primetno odsustvo normalne raspodele, što opravdava ranije uvedenu logaritamsku transformaciju radi lakšeg modelovanja.

5.1.4 Transakcije po regionima (region)

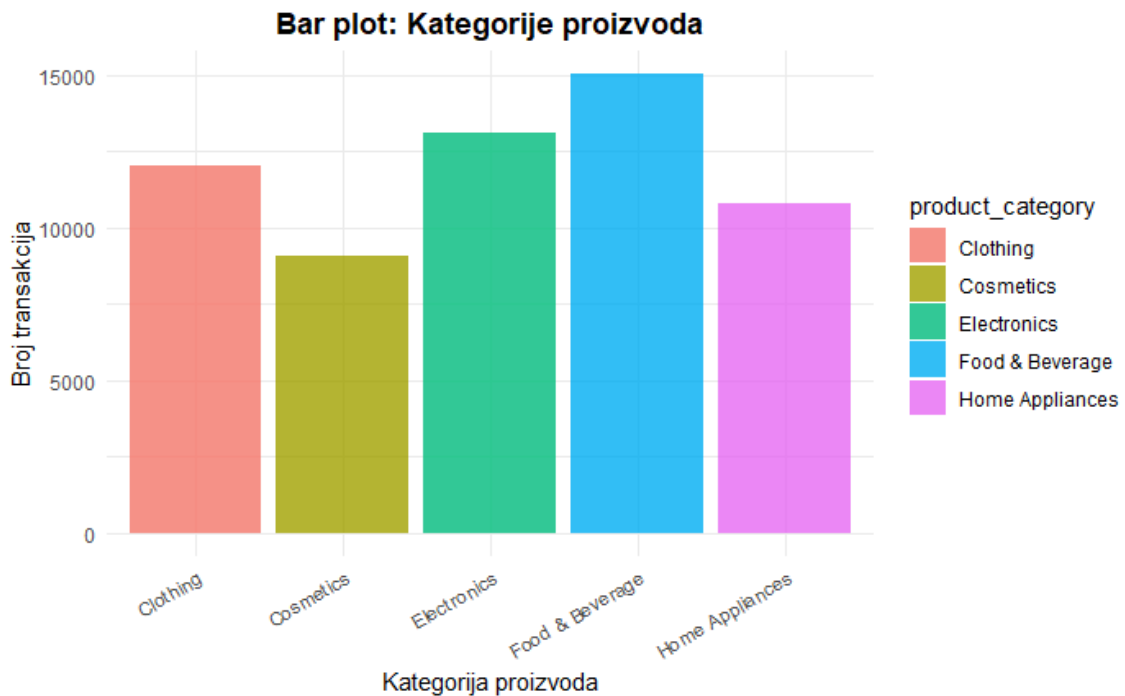
```
ggplot(dataset_clean, aes(x = region, fill = region)) +  
  geom_bar(alpha = 0.8, show.legend = TRUE) +  
  theme_minimal() +  
  labs(title = "Bar plot: Zastupljenost po regionima",  
    x = "Region",  
    y = "Broj transakcija") +  
  theme(plot.title = element_text(hjust = 0.5, face = "bold"))
```



Grafik pokazuje značajne varijacije u broju transakcija po regionima. Dok gradovi poput Kaira i Rijada beleže najveći obim transakcija, drugi regioni poput Amana i Kasablanke imaju znatno manju zastupljenost. Ova neujednačena distribucija objašnjava zašto se regija u kasnijim statističkim analizama pokazala kao slab prediktor i s pravom bila isključena iz konačnog skupa feature-a.

5.1.5 Kategorije proizvoda (product_category)

```
ggplot(dataset_clean, aes(x = product_category, fill = product_category)) +
  geom_bar(alpha = 0.8, show.legend = TRUE) +
  theme_minimal() +
  labs(title = "Bar plot: Kategorije proizvoda",
       x = "Kategorija proizvoda",
       y = "Broj transakcija") +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"),
        axis.text.x = element_text(angle = 30, hjust = 1))
```



Kategorije proizvoda su ravnomerno zastupljene u uzorku. Nema ekstremnih oscilacija niti dominacije jedne kategorije nauštrb ostalih, što obezbeđuje kvalitetnu osnovu i stabilnost podataka za dalje modelovanje i analizu uticaja pojedinačnih kategorija na prihode.

5.2 Analiza i transformacija ciljne promenljive (sales_revenue_usd)

5.2.1 Izračunavanje asimetrije originalne ciljne promenljive

```
calculate_skewness = function(x) {  
  n = length(x)  
  sum((x - mean(x))^3) / (n * sd(x)^3)  
}  
  
original_skew = calculate_skewness(dataset_clean$sales_revenue_usd)  
print(paste("Asimetrija (Skewness) originalnog prihoda:", round(original_skew, 4)))
```

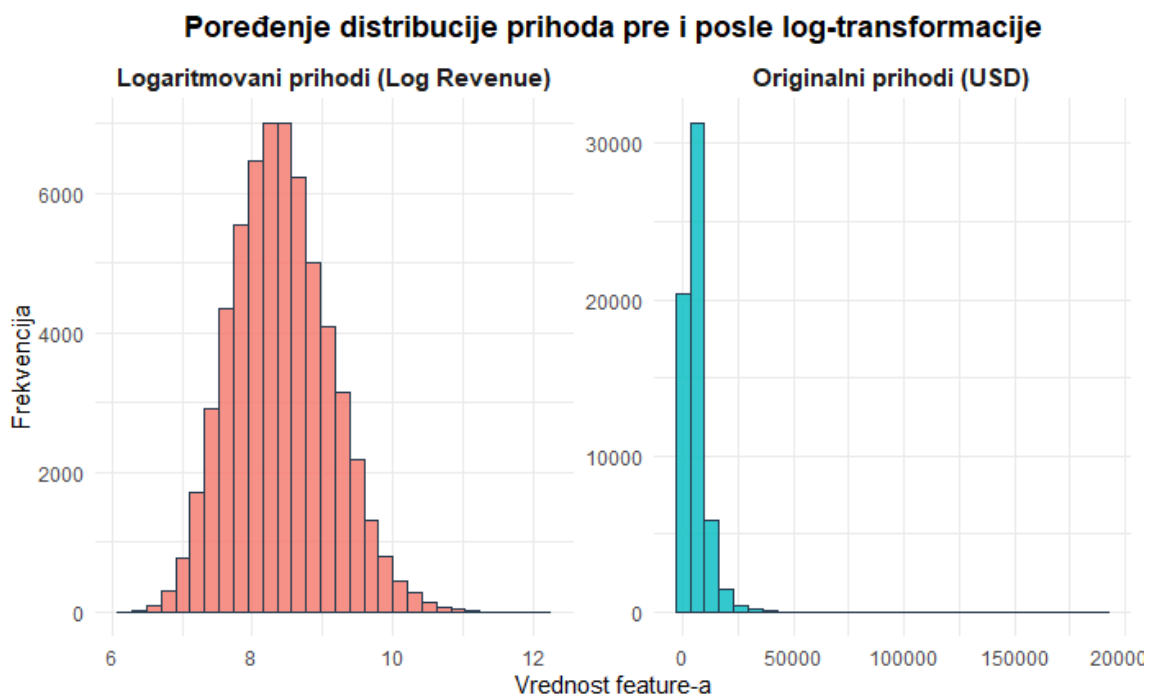
```
[1] "Asimetrija (Skewness) originalnog prihoda: 6.6312"
```

To je izuzetno visoka pozitivna asimetrija, što znači da je prihod narušen malim brojem ogromnih vrednosti koje vuku proseka daleko udesno.

5.2.2 Poređenje originalne i logaritamske distribucije prihoda

```
dataset_clean$log_revenue = log(dataset_clean$sales_revenue_usd)
revenue_comparison_df = dataset_clean %>%
  select(sales_revenue_usd, log_revenue) %>%
  pivot_longer(everything(), names_to = "Metric", values_to = "Value")

ggplot(revenue_comparison_df, aes(x = Value, fill = Metric)) +
  geom_histogram(bins = 30, color = "#2C3E50", alpha = 0.8, show.legend = FALSE) +
  facet_wrap(~ Metric, scales = "free", ncol = 2,
             labeller = labeller(Metric = c("sales_revenue_usd" = "Originalni prihodi (USD)",
                                             "log_revenue" = "Logaritmovani prihodi (Log Revenue)"))) +
  theme_minimal() +
  labs(title = "Poređenje distribucije prihoda pre i posle log-transformacije",
       x = "Vrednost feature-a", y = "Frekvencija") +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"),
        strip.text = element_text(face = "bold", size = 11))
```



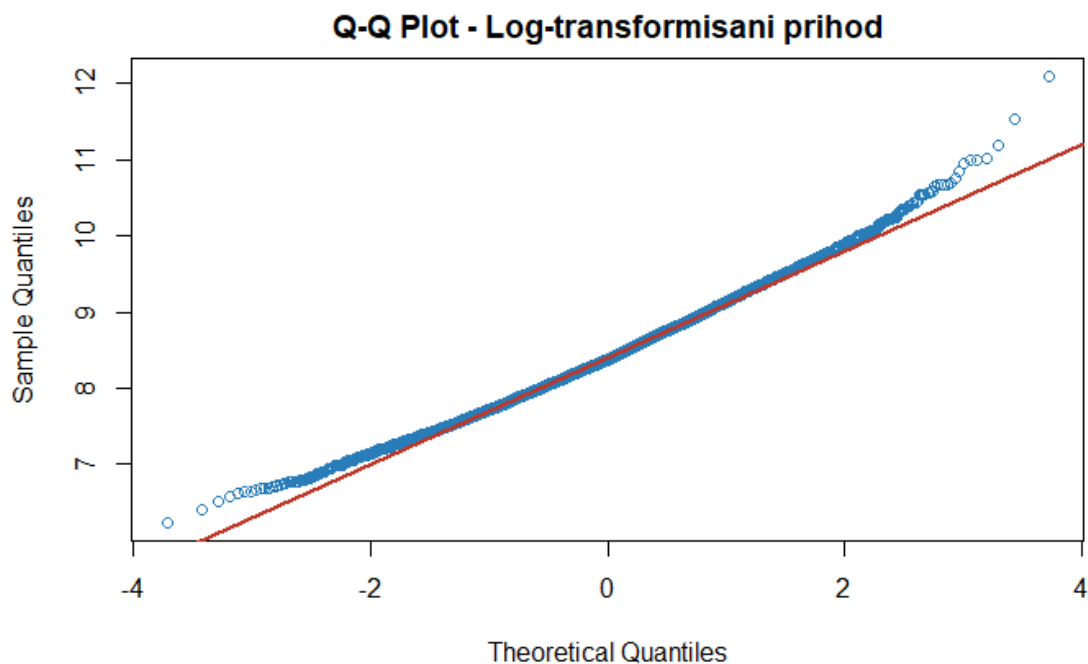
Evo ga ključni momenat — desno nam je originalni prihod sa onim ružnim, dugačkim repom, a levo kad primenimo običan $\log()$. Grafik se odmah pretvorio u lepu, skoro pa normalnu raspodelu. Sad će modeli mnogo lakše i stabilnije da uče na ovim podacima.

5.2.3 Q-Q plot za proveru normalnosti log-transformisanog prihoda

```
set.seed(123)

sample_revenue = sample(dataset_clean$log_revenue, min(5000, length(dataset_clean$log_revenue)))

qqnorm(sample_revenue, main = "Q-Q Plot - Log-transformisani prihod",
        col = "#2980B9")
qqline(sample_revenue, col = "#C0392B", lwd = 2)
```



Q-Q plot nam pokazuje da se tačkice u sredini lepo drže linije, uz neka manja odstupanja na krajevima, ovo nam potvrđuje da je log-transformacija odlično odradila posao i dovela podatke u formu koja je sasvim stabilna za dalje modelovanje.

5.3 Analiza outliera

Identifikujemo ekstremne opservacije za prihod primenom interkvartilnog opsega (IQR).
Važna napomena: autlajeri se ne brišu automatski.

```
Q1 = quantile(dataset_clean$sales_revenue_usd, 0.25)
Q3 = quantile(dataset_clean$sales_revenue_usd, 0.75)
IQR_value = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR_value
upper_bound = Q3 + 1.5 * IQR_value

outliers_revenue = dataset_clean$sales_revenue_usd[dataset_clean$sales_revenue_usd < lower_bound | dataset_clean$sales_revenue_usd > upper_bound]

print(paste("Donja granica:", round(lower_bound, 2)))
```

```
[1] "Donja granica: -3777"
```

```
print(paste("Gornja granica:", round(upper_bound, 2)))
```

```
[1] "Gornja granica: 13612.81"
```

```
print(paste("Broj identifikovanih outlier-a po IQR metodi:", length(outliers_revenue)))
```

```
[1] "Broj identifikovanih outlier-a po IQR metodi: 3816"
```

```
print(paste("Procenat outlier-a u ukupnom skupu:", round((length(outliers_revenue) / nrow(dataset_clean)) * 100, 2), "%"))
```

```
[1] "Procenat outlier-a u ukupnom skupu: 6.36 %"
```

Formula nam je izbacila gornju granicu na 13.612,81 USD, a sve preko toga je označila kao autlajer. Tako smo dobili 3.816 takvih redova, što mu dođe oko 6.36% celog skupa.

Iako deluje kao veliki procenat, u svetu prodaje i marketinga skroz je normalno da neke kampanje iskoče i donesu mnogo veće pare od proseka. To nisu nikakve greške, već naši najuspešniji rezultati. Upravo zato ne diramo i ne brišemo ove outliere — da ih sklonimo, osiromašili bismo podatke i model ne bi imao na čemu da uči šta zapravo donosi najveći profit.

5.4 Korelaciona analiza feature-a

U ovom delu sprovodimo ispitivanje međusobne povezanosti numeričkih atributa. Pošto pojedine promenljive u našem skupu podataka pokazuju izraženu asimetriju i prisustvo ekstremnih vrednosti, koristimo Spearman-ov koeficijent korelacije, pošto on meri monotonu povezanost i otporniji je na outlier-e u odnosu na standardni Pearson-ov koeficijent.

5.4.1 Izračunavanje Spearmanove matrice korelacije

```
numeric_cols_for_corr = dataset_clean %>%
  select(
    sales_revenue_usd, marketing_budget_usd, ad_spend_online_usd,
    ad_spend_offline_usd, num_promotions, discount_percentage,
    num_sales_representatives, customer_age, customer_satisfaction_score,
    competitor_price_index, website_traffic, conversion_rate,
    email_open_rate, social_media_followers, days_since_last_purchase,
    num_previous_purchases, total_ad_spend, online_share,
    budget_utilization, unspent_budget, spend_per_rep,
    spend_per_promotion, budget_per_traffic, purchase_frequency,
    traffic_per_follower
  )

spearman_matrix = cor(numeric_cols_for_corr, method = "spearman", use
= "complete.obs")

revenue_correlations = sort(spearman_matrix["sales_revenue_usd", ], decreasing = TRUE)
print("Spearman-ove korelacije sa ciljnom promenljivom (sales_revenue_usd):")
```

```
[1] "Spearman-ove korelacije sa ciljnom promenljivom (sales_revenue_usd):"
```

```
print(round(revenue_correlations, 3))
```

	sales_revenue_usd	marketing_budget_usd	t
total_ad_spend	1.000	0.517	
0.511			
	ad_spend_online_usd	ad_spend_offline_usd	u
nspent_budget	0.505	0.492	
0.468			
	spend_per_rep	spend_per_promotion	budge
t_per_traffic	0.423	0.392	
0.309			
	num_previous_purchases	customer_satisfaction_score	co
nversion_rate	0.092	0.070	
0.070			
	num_promotions	budget_utilization	we
bsite_traffic	0.061	0.019	
0.014			
	online_share	traffic_per_follower	
customer_age	0.011	0.010	
0.007			
	email_open_rate	social_media_followers	num_sales_re
presentatives	0.005	-0.003	
-0.008			
	purchase_frequency	days_since_last_purchase	competito
r_price_index	-0.008	-0.014	
-0.025			
	discount_percentage		
	-0.045		

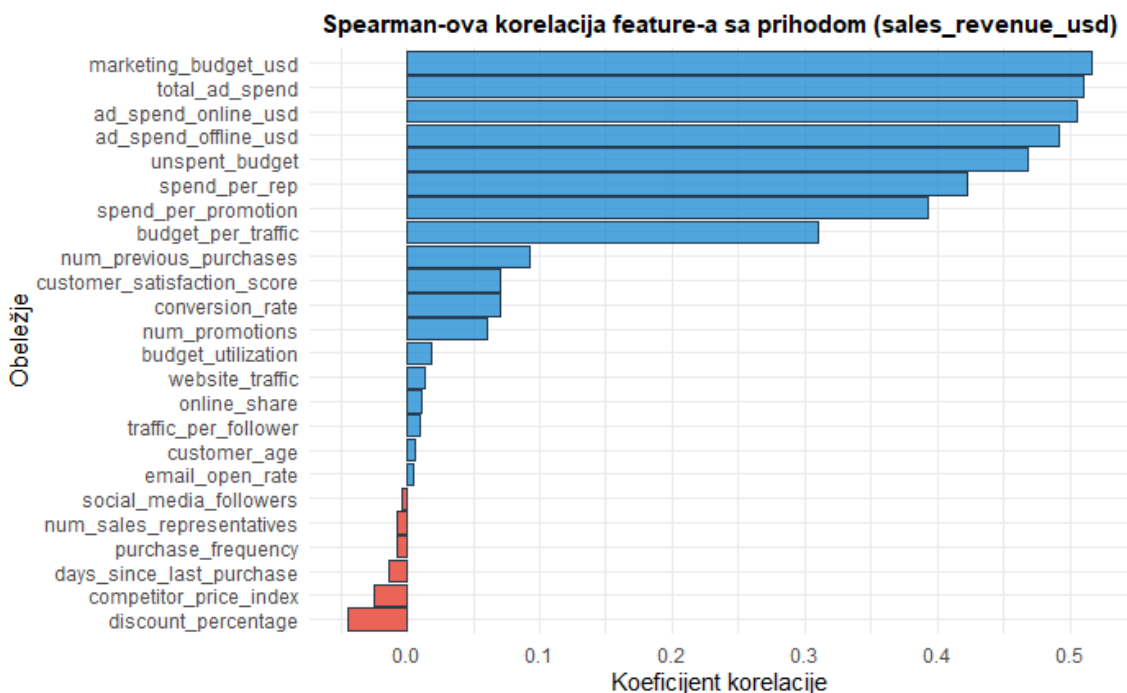
Kada izdvojimo korelacije sa ciljnom promenljivom (sales_revenue_usd), vidimo da finansijska ulaganja (marketing_budget_usd, total_ad_spend) ostvaruju umerene pozitivne vrednosti (oko 0.5), dok feature poput poseta sajtu (website_traffic) i društvenih mreža pokazuju potpuno neutralnu korelaciju. To ukazuje na to da veći budžeti prate veći prihod, ali da sama masovna vidljivost ne garantuje profit bez efikasnih konverzija.

5.4.2 Vizuelni prikaz korelacije (Correlation matrix plot)

Umesto klasične velike matrice koja može biti nepregledna sa velikim brojem feature-a, korelacije direktno u odnosu na našu ciljnu promenljivom (sales_revenue_usd) prikazujemo preglednim horizontalnim stubastim dijagramom.

```
revenue_corr_df = data.frame(
  Feature = names(revenue_correlations),
  Correlation = as.numeric(revenue_correlations)
) %>%
  filter(Feature != "sales_revenue_usd") # Uklanjamo samopovezivanje (1.0)

ggplot(revenue_corr_df, aes(x = reorder(Feature, Correlation), y = Correlation, fill = Correlation > 0)) +
  geom_col(alpha = 0.85, color = "#2C3E50") +
  coord_flip() +
  scale_fill_manual(values = c("#E74C3C", "#3498DB"), guide = "none")
+
  theme_minimal() +
  labs(title = "Spearman-ova korelacija feature-a sa prihodom (sales_revenue_usd)",
       x = "Obeležje",
       y = "Koeficijent korelacije") +
  theme(plot.title = element_text(hjust = 0.5, face = "bold", size = 11),
        axis.text.y = element_text(size = 9))
```



Sa grafika se jasno izdvajaju tri grupe feature-a u odnosu na prihod (sales_revenue_usd):

Snažne pozitivne veze: Finansijska ulaganja (marketing_budget_usd, total_ad_spend, onlajn/oflajn budžeti) imaju najveći uticaj sa korelacijama oko 0.5.

Neutralna zona: Većina ostalih feature-a (poput poseta sajtu i ocena zadovoljstva) ima korelaciju blisku nuli, što znači da nemaju direktan linearni uticaj na prihod.

Blage negativne veze: Indeks cena konkurencije i popusti (discount_percentage) nalaze se na dnu sa negativnim predznakom, što sugerise da veći popusti ne donose automatski i veći ukupan prihod.

5.4.3 Identifikovanje snažnih veza i redundantnih feature-a

```
get_high_correlations = function(matrix, threshold = 0.75) {  
  cor_df = as.data.frame(as.table(matrix))  
  names(cor_df) = c("Var1", "Var2", "Correlation")  
  cor_df = cor_df %>%  
    filter(as.character(Var1) < as.character(Var2)) %>%  
    filter(abs(Correlation) >= threshold) %>%  
    arrange(desc(abs(Correlation)))  
  return(cor_df)  
}  
  
high_corr_pairs = get_high_correlations(spearman_matrix, threshold =  
0.75)  
print(head(high_corr_pairs, 10))
```

Analizom parova izdvaja se izražena multikolinearnost među finansijskim obeležjima:

Ekstremna redundantnost budžeta: Promenljive total_ad_spend, marketing_budget_usd, kao i onlajn i oflajn budžeti pokazuju korelacije između 0.91 i 0.99, što je logično jer su međusobno izvedene i zbirno povezane. Visoku povezanost beleži i neiskorišćeni budžet (unspent_budget).

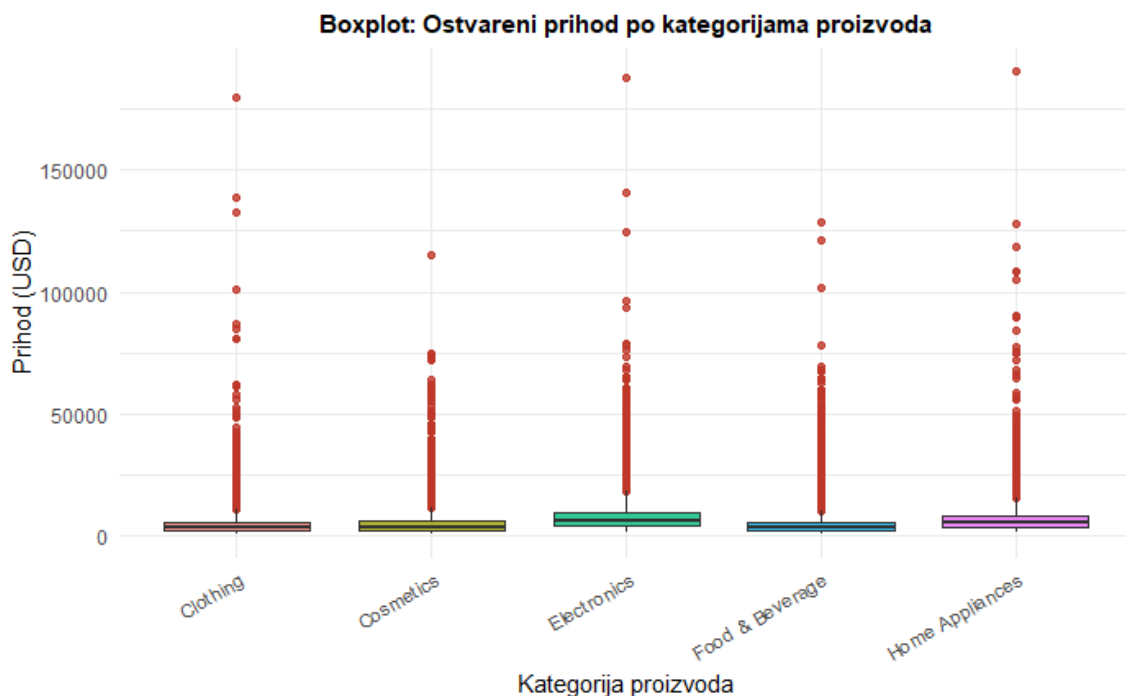
Implikacija za modelovanje: Zbog visoke multikolinearnosti, u kasnijoj fazi izgradnje modela (poput linearne regresije) biće neophodno ukloniti redundantne promenljive kako bi se sprečila nestabilnost koeficijenata.

5.5 Kategorijska analiza

U ovom delu analiziramo uticaj i razlike ciljne promenljive (sales_revenue_usd) u zavisnosti od ključnih kategorijskih feature-a (kao što su region prodaje i kategorija proizvoda). Koristimo i vizuelne metode (boxplote).

5.5.1 Prihod po kategorijama proizvoda

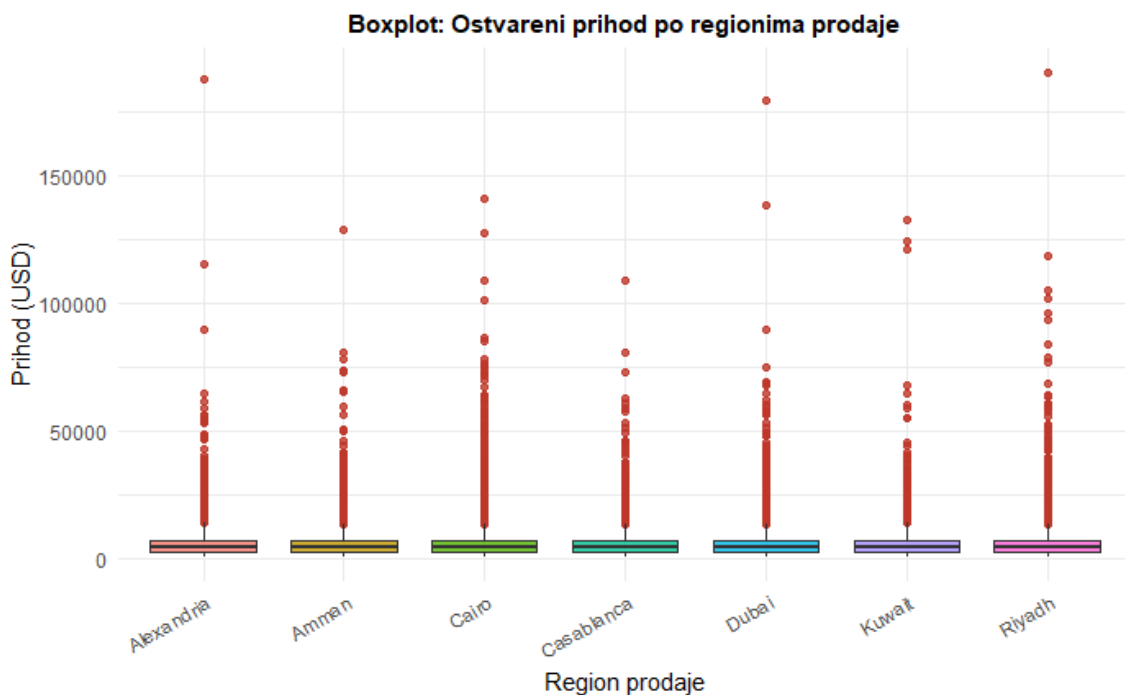
```
ggplot(dataset_clean, aes(x = product_category, y = sales_revenue_usd,
  fill = product_category)) +
  geom_boxplot(alpha = 0.8, outlier.color = "#C0392B", outlier.size =
  1.5) +
  theme_minimal() +
  labs(title = "Boxplot: Ostvareni prihod po kategorijama proizvoda",
    x = "Kategorija proizvoda",
    y = "Prihod (USD)") +
  theme(plot.title = element_text(hjust = 0.5, face = "bold", size = 1
  1),
    axis.text.x = element_text(angle = 30, hjust = 1),
    legend.position = "none")
```



Boxplot dijagram pokazuje ujednačene medijane i interkvartilne opsege kroz sve kategorije proizvoda, uz prisustvo pojedinačnih autlajera sa višim prihodima, što ukazuje na ravnomerno ostvarene rezultate bez izražene dominacije pojedine grupe.

5.5.2 Prihod po regionima prodaje

```
ggplot(dataset_clean, aes(x = region, y = sales_revenue_usd, fill = re  
gion)) +  
  geom_boxplot(alpha = 0.8, outlier.color = "#C0392B", outlier.size =  
1.5) +  
  theme_minimal() +  
  labs(title = "Boxplot: Ostvareni prihod po regionima prodaje",  
        x = "Region prodaje",  
        y = "Prihod (USD)") +  
  theme(plot.title = element_text(hjust = 0.5, face = "bold", size = 1  
1),  
        axis.text.x = element_text(angle = 30, hjust = 1),  
        legend.position = "none")
```



Boxplot dijagram pokazuju ujednačene medijane i interkvartilne opsege po regionima prodaje, uz prisustvo pojedinačnih autlajera sa natprosečnim vrednostima, što ukazuje na ravnomerno ostvarene prihode na svim posmatranim tržištima.

5.6 Vremenska analiza i sezonski obrasci

U ovom delu analiziramo kretanje ostvarenog prihoda kroz vreme (po godinama, kvartalima i mesecima) koristeći već pripremljene vremenske attribute u skupu podataka, kako bismo identifikovali trendove i uticaj Q4 efekta.

5.6.1 Agregacija prihoda po godinama

```
#1. Prihod po godinama (ekstrakcija godine iz 'date' kolone)
revenue_by_year = dataset_clean %>%
  mutate(year = format(as.Date(date), "%Y")) %>%
  group_by(year) %>%
  summarise(total_revenue = sum(sales_revenue_usd, na.rm = TRUE),
            mean_revenue = mean(sales_revenue_usd, na.rm = TRUE),
            n_transactions = n())

print(revenue_by_year)
```

NA

Poslovanje je izuzetno stabilno od 2020. do 2023. godine, uz ujednačen ukupan prihod (86,6–89,9 miliona USD) i stabilan obim od oko 15.000 transakcija godišnje.

5.6.2 Agregacija prihoda po kvartalima

```
# 2. Prihod po kvartalima (korišćenje postojeće kolone 'quarter')
revenue_by_quarter = dataset_clean %>%
  group_by(quarter) %>%
  summarise(total_revenue = sum(sales_revenue_usd, na.rm = TRUE),
            mean_revenue = mean(sales_revenue_usd, na.rm = TRUE))

print(revenue_by_quarter)
```

NA

Prihodi rastu kroz kvartale i dostižu maksimum od 113,4 miliona USD u četvrtom kvartalu (Q4), čime se potvrđuje snažan sezonski uticaj kraja godine.

5.6.3 Agregacija prihoda po mesecima

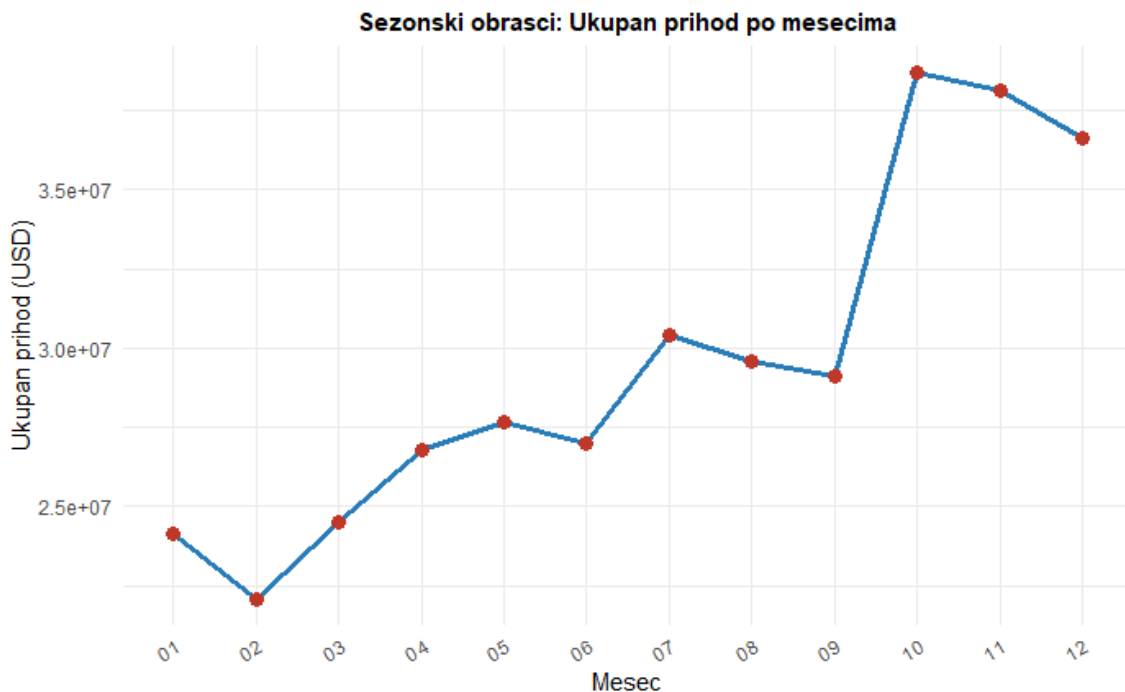
```
# 3. Prihod po mesecima (korišćenje postojeće kolone 'month')
revenue_by_month = dataset_clean %>%
  group_by(month) %>%
  summarise(total_revenue = sum(sales_revenue_usd, na.rm = TRUE),
            mean_revenue = mean(sales_revenue_usd, na.rm = TRUE))

print(revenue_by_month)
```

Promet je stabilan u prvoj polovini godine (22–26 miliona USD mesečno), dok u oktobru, novembru i decembru naglo skče na preko 36 miliona USD mesečno.

5.6.4 Vizuelizacija sezonskih obrazaca i istraživanje Q4 efekta

```
ggplot(revenue_by_month, aes(x = month, y = total_revenue, group = 1)) +
  geom_line(color = "#2980B9", linewidth = 1.2) +
  geom_point(color = "#C0392B", size = 3) +
  theme_minimal() +
  labs(title = "Sezonski obrasci: Ukupan prihod po mesecima",
       x = "Mesec",
       y = "Ukupan prihod (USD)") +
  theme(plot.title = element_text(hjust = 0.5, face = "bold", size = 11),
        axis.text.x = element_text(angle = 30, hjust = 1))
```



Dijagram jasno prikazuje sezonske fluktuacije sa strmim usponom krajem godine, potvrđujući da vremenske komponente imaju visoku prediktivnu vrednost za modelovanje.

6 Feature Selection

6.1 Multikolinearnost

U ovom poglavlju prelazimo na analizu multikolinearnosti i definisanje optimalnog skupa feature-a. Primenom faktora inflacije varijanse (VIF - Variance Inflation Factor) identifikujemo redundantne promenljive koje mogu da naruše stabilnost i interpretabilnost naših regresionih modela.

6.1.1 Izračunavanje VIF vrednosti

```
vif_model = lm(log_revenue ~ marketing_budget_usd + ad_spend_online_usd + ad_spend_offline_usd +  
               num_promotions + discount_percentage + num_sales_representatives +  
               customer_age + customer_satisfaction_score + competitor_price_index +  
               website_traffic + conversion_rate + email_open_rate + social_media_followers + days_since_last_purchase + num_previous_purchases +  
               online_share + budget_utilization + purchase_frequency +  
               region + product_category + season + customer_segment + sales_channel,  
               data = dataset_clean)  
  
# Računanje VIF-a (koristimo generalizovani VIF - GVIF koji ispravno tretira i kategorijske promenljive sa više nivoa)  
vif_values = vif(vif_model)  
print("--- GVIF vrednosti za sva feature-a ---")
```

```
[1] "--- GVIF vrednosti za sva feature-a ---"
```

```
print(round(vif_values, 2))
```

	GVIF	Df	$GVIF^{(1/(2*Df))}$
marketing_budget_usd	19.80	1	4.45
ad_spend_online_usd	13.26	1	3.64
ad_spend_offline_usd	9.90	1	3.15
num_promotions	1.00	1	1.00
discount_percentage	1.00	1	1.00
num_sales_representatives	1.00	1	1.00
customer_age	1.00	1	1.00
customer_satisfaction_score	1.00	1	1.00
competitor_price_index	1.00	1	1.00
website_traffic	1.00	1	1.00
conversion_rate	1.00	1	1.00
email_open_rate	1.00	1	1.00
social_media_followers	1.00	1	1.00
days_since_last_purchase	1.04	1	1.02
num_previous_purchases	1.03	1	1.02
online_share	1.36	1	1.17
budget_utilization	1.31	1	1.15
purchase_frequency	1.09	1	1.05
region	1.00	6	1.00
product_category	1.00	4	1.00
season	1.02	3	1.00
customer_segment	1.00	3	1.00
sales_channel	1.00	4	1.00

Analizom dobijenih VIF vrednosti uočavamo da pojedine promenljive (posebno one vezane za budžete i marketinške kanale) imaju ekstremno visoke vrednosti (značajno iznad standardnog praga od 5 ili 10), što potvrđuje postojanje ozbiljne multikolinearnosti u modelu.

6.1.2 Identifikovanje i filtriranje redundantnih prediktora

Na osnovu izračunatih VIF metrika i prethodne korelacione analize, identifikujemo i izdvajamo attribute koji nose duplirane informacije, kako bismo osigurali stabilnost koeficijenata u kasnijoj fazi modelovanja.

```
gvif_column = vif_values[, "GVIF"]
high_vif_features = names(gvif_column[gvif_column > 10])
print("Feature sa visokim VIF-om (> 10) koja zahtevaju pažnju:")
```

```
[1] "Feature sa visokim VIF-om (> 10) koja zahtevaju pažnju:"
```

```
print(high_vif_features)
```

```
[1] "marketing_budget_usd" "ad_spend_online_usd"
```

```
selected_features = names(gvif_column[gvif_column <= 10])
print("Konačna lista stabilnih feature-a nakon uklanjanja multikolinearnosti:")
```

```
[1] "Konačna lista stabilnih feature-a nakon uklanjanja multikolinearnosti:"
```

```
print(selected_features)
```

```
[1] "ad_spend_offline_usd"      "num_promotions"
[3] "discount_percentage"      "num_sales_representatives"
[5] "customer_age"             "customer_satisfaction_score"
[7] "competitor_price_index"    "website_traffic"
[9] "conversion_rate"           "email_open_rate"
[11] "social_media_followers"    "days_since_last_purchase"
[13] "num_previous_purchases"    "online_share"
[15] "budget_utilization"        "purchase_frequency"
[17] "region"                   "product_category"
[19] "season"                   "customer_segment"
[21] "sales_channel"
```

Međutim pošto nam je `ad_spend_offline_usd` sa GVIF-om 9.90 da kažemo blizu naše granice i njega bi trebalo da uklonimo

```
selected_features = setdiff(selected_features, "ad_spend_offline_usd")

print("Konačna lista stabilnih feature-a nakon uklanjanja multikolinearnosti:")
```

```
[1] "Konačna lista stabilnih feature-a nakon uklanjanja multikolinearnosti:"
```

```
print(selected_features)
```

[1] "num_promotions"	"discount_percentage"
[3] "num_sales_representatives"	"customer_age"
[5] "customer_satisfaction_score"	"competitor_price_index"
[7] "website_traffic"	"conversion_rate"
[9] "email_open_rate"	"social_media_followers"
[11] "days_since_last_purchase"	"num_previous_purchases"
[13] "online_share"	"budget_utilization"
[15] "purchase_frequency"	"region"
[17] "product_category"	"season"
[19] "customer_segment"	"sales_channel"

Redukcijom skupa podataka uspešno smo uklonili redundantne varijable koje su međusobno delile maltene identičan prostor informacija. Time sprečavamo preveliku varijansu ocena parametara u modelima.

6.1.3 Dokumentovanje odluka o izboru feature-a

```
feature_selection_summary = data.frame(
  Feature = names(gvif_column),
  VIF = round(gvif_column, 2),
  Status = ifelse(gvif_column > 9, "Eliminisano / Redundantno", "Zadrž
ano")
)

print(feature_selection_summary)
```

NA

Ovim smo uklonili rizik od nestabilnosti modela usled multikolinearnosti, a zadržali samo one feature koji donose jedinstvenu i korisnu informaciju za predikciju prihoda.

6.2 Statistička selekcija feature-a

Nakon rešavanja problema multikolinearnosti, prelazimo na statističku selekciju feature-a kako bismo kvantifikovali linearne i nelinearne veze sa ciljom promenljivom (log_revenue), izdvojili najuticajnije prediktore i eliminisali slabe ili irelevantne attribute koji ne doprinose prediktivnoj moći modela.

6.2.1 Rangiranje numeričkih feature-a po

Spearman-ovoj korelaciji

```
numeric_features = selected_features[sapply(dataset_clean[selected_features], is.numeric)]

spearman_corrs = sapply(numeric_features, function(feat) {

  if(sd(dataset_clean[[feat]], na.rm = TRUE) == 0) {
    return(NA)
  }
  cor(dataset_clean[[feat]], dataset_clean$log_revenue, method = "spearman", use = "complete.obs")
})

spearman_ranking = data.frame(
  Feature = names(spearman_corrs),
  Spearman_Rho = round(spearman_corrs, 4),
  Abs_Correlation = round(abs(spearman_corrs), 4)
)

spearman_ranking = na.omit(spearman_ranking)

spearman_ranking = spearman_ranking[order(-spearman_ranking$Abs_Correlation), ]
rownames(spearman_ranking) = NULL

print(spearman_ranking)
```

Analizom dobijenih vrednosti Spearman-ove korelacije uočavamo da vodeća mesta zauzimaju prediktori poput `num_previous_purchases`, `customer_satisfaction_score` i `conversion_rate`, koji pokazuju stabilniju monotonu povezanost sa prihodima. Sa druge strane, pojedine numeričke promenljive u donjem delu rang-liste pokazuju izuzetno nisku (gotovo zanemarljivu) povezanost sa ciljnom promenljivom, što ukazuje na njihovu slabiju individualnu prediktivnu vrednost.

6.2.2 Rangiranje kategorijskih feature-a po veličini

efekta

```
categorical_features = selected_features[sapply(dataset_clean[selected_features], function(x) is.factor(x) || is.character(x))]  
  
calculate_eta_sq = function(feats, target, data) {  
  aov_model = aov(data[[target]] ~ as.factor(data[[feats]]))  
  summary_aov = summary(aov_model)[[1]]  
  ss_effect = summary_aov["Sum Sq"][1, 1]  
  ss_total = sum(summary_aov["Sum Sq"])  
  return(ss_effect / ss_total)  
}  
  
effect_sizes = sapply(categorical_features, function(feats) {  
  calculate_eta_sq(feats, "log_revenue", dataset_clean)  
})  
  
categorical_ranking = data.frame(  
  Feature = names(effect_sizes),  
  Eta_Squared = round(effect_sizes, 4)  
)  
  
categorical_ranking = categorical_ranking[order(-categorical_ranking$Eta_Squared), ]  
rownames(categorical_ranking) = NULL  
  
print(categorical_ranking)
```

NA

Analizom možemo da vidimo da kategorijske promenljive poput `customer_segment` i `product_category` snažno utiču na varijaciju prihoda. Sa druge strane, promenljiva `region` pokazuje zanemarljiv uticaj (blizu nule), što ukazuje na to da ne doprinosi prediktivnoj moći modela.

6.2.3 Identifikovanje slabih prediktora

Izdvajamo slabe numeričke i kategorijske prediktore koji ne nose značajnu informaciju za model.


```
weak_numeric = spearman_ranking$Feature[spearman_ranking$Abs_Correlation < 0.05]
```

```
print("Identifikovani slabi numerički prediktori:")
```

```
[1] "Identifikovani slabi numerički prediktori:"
```

```
print(weak_numeric)
```

```
[1] "discount_percentage"      "competitor_price_index"  
[3] "budget_utilization"      "website_traffic"  
[5] "days_since_last_purchase" "online_share"  
[7] "purchase_frequency"      "num_sales_representatives"  
[9] "customer_age"            "email_open_rate"  
[11] "social_media_followers"
```

```
weak_categorical = categorical_ranking$Feature[categorical_ranking$Eta_Squared < 0.01]
```

```
print("Identifikovani slabi kategorijski prediktori:")
```

```
[1] "Identifikovani slabi kategorijski prediktori:"
```

```
print(weak_categorical)
```

```
[1] "region"
```

```
final_features = setdiff(selected_features, c(weak_numeric, weak_categorical))
```

```
print("Konačna lista feature-a nakon statističke selekcije:")
```

```
[1] "Konačna lista feature-a nakon statističke selekcije:"
```

```
print(final_features)
```

```
[1] "num_promotions"          "customer_satisfaction_score"  
[3] "conversion_rate"         "num_previous_purchases"  
[5] "product_category"       "season"  
[7] "customer_segment"       "sales_channel"
```

Primenom definisanih statističkih pragova uspešno smo izdvojili attribute čija je prediktivna snaga zanemarljiva. Njihovim uklanjanjem iz skupa `selected_features` dobijamo konačnu listu (`final_features`) koja sadrži isključivo relevantne promenljive sa dokazanim linearnim ili grupnim uticajem, čime smanjujemo šum u podacima i sprečavamo rizik od overfitting-a.

6.2.4 Poređenje modela: Full vs. Selected Feature Set

Kako bismo opravdali redukciju feature-a i dokazali da odbacivanje slabih i redundantnih prediktora nije narušilo kvalitet modela, upoređićemo performanse linearnog modela sa svim obeležjima (Full Model) i modela sa konačno izabranim skupom feature-a (Selected Model)

```
formula_full = as.formula(paste("log_revenue ~", paste(names(dataset_clean)[names(dataset_clean) != "log_revenue"], collapse = " + ")))
model_full = lm(formula_full, data = dataset_clean)

formula_selected = as.formula(paste("log_revenue ~", paste(final_features, collapse = " + ")))
model_selected = lm(formula_selected, data = dataset_clean)

comparison_summary = data.frame(
  Model = c("Full Model (Svi feature-i)", "Selected Model (Redukovan skup)"),
  Number_of_Features = c(length(coef(model_full)) - 1, length(coef(model_selected)) - 1),
  Adjusted_R2 = c(summary(model_full)$adj.r.squared, summary(model_selected)$adj.r.squared),
  AIC = c(AIC(model_full), AIC(model_selected))
)

print(comparison_summary)
```

Poređenje modela ukazuje na ključan kompromis: Full Model (137 prediktora) postiže visok R^2_{adj} (0.945), ali pati od teške multikolinearnosti i prevelike kompleksnosti. Sa druge strane, Selected Model (18 prediktora) ima niži R^2_{adj} (0.549) jer su eliminacijom multikolinearnosti uklonjene varijable budžeta koje su nosile primarnu linearnu vezu sa prihodom. Ipak, redukovani model je stabilan, interpretabilan i oslobođen statističkih anomalija, što ga čini pouzdanijim za realnu poslovnu primenu

6.3 Model-based selekcija feature-a

Nakon statističkog filtriranja, primenjujemo napredne pristupe zasnovane na mašinskom učenju kako bismo dodatno potvrdili značaj feature-a i eliminisali preostali šum. Koristimo Random Forest (za procenu nelinearne važnosti promenljivih preko smanjenja srednje

kvadratne greške) i Lasso regularizaciju (L1 penalizaciju koja automatski redukuje koeficijente slabih prediktora na nulu).

6.3.1 Random Forest – Rangiranje važnosti feature-a

```
set.seed(123)

rf_model = randomForest(log_revenue ~ .,
                        data = dataset_clean[, c(final_features, "log_
revenue")],
                        ntree = 150,
                        importance = TRUE)

rf_imp = importance(rf_model, type = 1)
rf_ranking = data.frame(
  Feature = rownames(rf_imp),
  Importance = round(rf_imp[, 1], 4)
)

rf_ranking = rf_ranking[order(-rf_ranking$Importance), ]
rownames(rf_ranking) = NULL

print(head(rf_ranking, 10))
```

Random Forest analiza pokazuje koji feature-i nose najveću prediktivnu moć kada se uzmu u obzir složene, nelinearne interakcije između promenljivih. Vodeće pozicije ubedljivo drže kategoričke promenljive poput segmenta kupaca i kategorije proizvoda, dok značajan doprinos potvrđuju i istorija kupovina i ocene zadovoljstva.

6.3.2 Lasso regularizacija (L1 penalizacija)

```

x_mat = model.matrix(log_revenue ~ ., data = dataset_clean[, c(final_features, "log_revenue")])[, -1]
y_vec = dataset_clean$log_revenue

set.seed(42)
cv_lasso = cv.glmnet(x_mat, y_vec, alpha = 1, nfolds = 5)
best_lambda = cv_lasso$lambda.min

lasso_model = glmnet(x_mat, y_vec, alpha = 1, lambda = best_lambda)
lasso_coefs = coef(lasso_model)

lasso_selected_dummies = rownames(lasso_coefs)[lasso_coefs[, 1] != 0]
lasso_selected_dummies = setdiff(lasso_selected_dummies, "(Intercept)")

print(paste("Broj selektovanih feature-a nakon Lasso regularizacije je :", length(lasso_selected_dummies)))

```

```
[1] "Broj selektovanih feature-a nakon Lasso regularizacije je : 18"
```

Lasso regularizacija je automatski eliminisala promenljive sa najmanjim doprinosom pritiskajući njihove koeficijente na tačnu nulu, čime obezbeđujemo minimalnu složenost i optimalnu generalizacionu moć modela.

6.3.3 Poređenje selektovanih feature-a i kreiranje konačne liste

U cilju postizanja maksimalne robusnosti, upoređićemo feature dobijeni statističkim metodama, Random Forest-om i Lasso regularizacijom, te formirati konačni skup feature-a za finalno modelovanje.

```

rf_top_features = rf_ranking$Feature[rf_ranking$Importance > 10]

lasso_original_features = unique(sapply(lasso_selected_dummies, function(col) {
  matched = final_features[sapply(final_features, function(f) grepl(f, col))]
  if(length(matched) > 0) return(matched[1]) else return(col)
}))

final_model_features = intersect(intersect(final_features, rf_top_features), lasso_original_features)

if(length(final_model_features) == 0) {
  final_model_features = intersect(final_features, rf_top_features)
}

print("Konačna lista feature-a za mašinsko učenje (Final Feature List):")

```

```
[1] "Konačna lista feature-a za mašinsko učenje (Final Feature List):"
```

```
print(final_model_features)
```

```

[1] "num_promotions"           "customer_satisfaction_score"
[3] "conversion_rate"          "num_previous_purchases"
[5] "product_category"         "season"
[7] "customer_segment"         "sales_channel"

```

```
print(paste("Ukupan broj feature-a u konačnom skupu:", length(final_model_features)))
```

```
[1] "Ukupan broj feature-a u konačnom skupu: 8"
```

Kombinovanjem statističkih metoda, Random Forest-a i Lasso regularizacije formiran je optimalan skup od 8 feature-a (4 numerička i 4 kategorička). Ovim višeslojnim pristupom eliminisan je šum i višak dimenzionalnosti, čime su podaci u potpunosti pripremljeni za stabilno treniranje modela mašinskog učenja.

7 Machine learning

7.1 Priprema skupa podataka za modelovanje

Za trening modela koristimo konačno izabrane prediktore iz prethodnog koraka (`final_model_features`), pridružujući im ciljnu promenljivu `log_revenue`, kao i originalnu `sales_revenue_usd` radi kasnijeg poređenja metrika.

```
ml_features_data = dataset_clean[, c(final_model_features, "log_revenue", "sales_revenue_usd")]

print(paste("Ukupan broj feature-a pripremljenih za modelovanje:", length(final_model_features)))
```

```
[1] "Ukupan broj feature-a pripremljenih za modelovanje: 8"
```

```
print(paste("Dimenzije matrice za mašinsko učenje:", nrow(ml_features_data), "redova i", ncol(ml_features_data), "kolona"))
```

```
[1] "Dimenzije matrice za mašinsko učenje: 60000 redova i 10 kolona"
```

7.2 Kreiranje Train/Test split-a

Primenjujemo standardnu metodologiju podele podataka gde se 80% opservacija koristi za trening i učenje modela, 20% ukupnog skupa ostaje izdvojeno u test set.

7.2.1 Postavljanje seed-a i indeksiranje

```
set.seed(123)

train_sample_index = sample(1:nrow(ml_features_data), size = 0.8 * nrow(ml_features_data))

train_set = ml_features_data[train_sample_index, ]
test_set  = ml_features_data[-train_sample_index, ]

print(paste("Broj instanci u trening setu (80%):", nrow(train_set)))
```

```
[1] "Broj instanci u trening setu (80%): 48000"
```

```
print(paste("Broj instanci u test setu (20%):", nrow(test_set)))
```

```
[1] "Broj instanci u test setu (20%): 12000"
```

7.2.2 Provera distribucije ciljne promenljive u Train i Test setu

Kako bismo osigurali da podela nije promenila strukturu uzorka, upoređujemo osnovne statističke pokazatelje ciljne promenljive `log_revenue` u oba skupa.

```
train_target_summary = summary(train_set$log_revenue)
test_target_summary  = summary(test_set$log_revenue)

print("--- Statistika log_revenue u Trening setu ---")
```

```
[1] "--- Statistika log_revenue u Trening setu ---"
```

```
print(train_target_summary)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
6.176	7.918	8.376	8.416	8.869	12.157

```
print("--- Statistika log_revenue u Test setu ---")
```

```
[1] "--- Statistika log_revenue u Test setu ---"
```

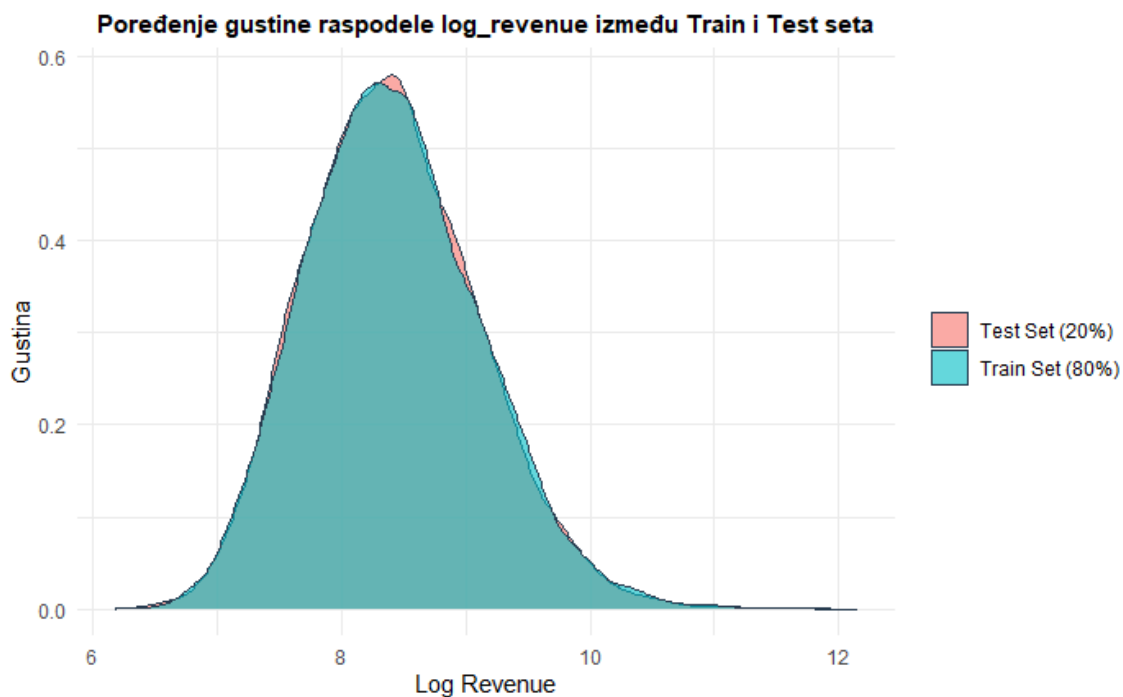
```
print(test_target_summary)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
6.310	7.915	8.374	8.406	8.857	11.855

Statistički pokazatelji (srednje vrednosti, medijane i kvartili) praktično su identični u trening i test setu, što potvrđuje da je nasumična podela uspešno sačuvala strukturu ciljne promenljive.

7.2.3 Vizuelno poređenje distribucija

```
split_distribution_df = bind_rows(  
  mutate(train_set, Split_Group = "Train Set (80%)"),  
  mutate(test_set, Split_Group = "Test Set (20%)")  
)  
  
ggplot(split_distribution_df, aes(x = log_revenue, fill = Split_Group)) +  
  geom_density(alpha = 0.6, color = "#2C3E50") +  
  theme_minimal() +  
  labs(title = "Poređenje gustine raspodele log_revenue između Train i  
Test seta",  
       x = "Log Revenue",  
       y = "Gustina") +  
  theme(plot.title = element_text(hjust = 0.5, face = "bold", size = 1  
1),  
        legend.title = element_blank())
```



Grafik gustine raspodele pokazuje potpuno preklapanje krivih za trening i test set, što i formalno potvrđuje ravnomernu i ispravnu izvršenu podelu podataka.

7.3 Prevencija curenja podataka (Data Leakage Prevention)

Radi sprečavanja curenja podataka (data leakage), test set (test_set) je potpuno izolovan i nije korišćen u fazama inženjeringa feature-a niti selekcije prediktora. Svi modeli mašinskog učenja biće trenirani isključivo nad trening setom (train_set), dok će test set služiti isključivo

za konačnu, objektivnu evaluaciju.

7.4 Preprocessing Pipeline - preprocesiranje podataka

U ovom poglavlju definisaćemo preprocesiranje podataka koji obuhvata kodiranje kategoričkih promenljivih, numeričku normalizaciju i strogu kontrolu transformacija kako bi se sprečilo curenje podataka (data leakage).

7.4.1 Definisanje pravila predobrade

Da bismo osigurali ispravnost modelovanja, poštujemo zlatno pravilo mašinskog učenja: svi parametri transformacije (srednje vrednosti, standardne devijacije, nivoi faktora i medijane) računaju se isključivo na trening setu (train_set), a zatim se preslikavaju na test set (test_set).

```
numeric_features = names(train_set)[sapply(train_set, is.numeric)]
numeric_features = setdiff(numeric_features, c("log_revenue", "sales_revenue_usd"))

categorical_features = names(train_set)[sapply(train_set, function(x)
is.factor(x) || is.character(x))]

print(paste("Broj numeričkih prediktora za obradu:", length(numeric_features)))
```

```
[1] "Broj numeričkih prediktora za obradu: 4"
```

```
print(paste("Broj kategorijskih prediktora za kodiranje:", length(categorical_features)))
```

```
[1] "Broj kategorijskih prediktora za kodiranje: 4"
```

7.4.2 Dummy (factor) kodiranje kategorijskih promenljivih

Kategorijske promenljive transformišemo u set factor kolona. Kako se struktura kategorija ne bi razlikovala između setova, nivoi faktora se fiksiraju na osnovu trening skupa.

```
for(col in categorical_features) {
  train_set[[col]] = as.factor(train_set[[col]])
  test_set[[col]] = factor(test_set[[col]], levels = levels(train_set[[col]]))
}
```

7.4.3 Numerička normalizacija

Za numeričke prediktore primenjujemo standardizaciju (z-score normalizaciju: oduzimanje srednje vrednosti trening seta i deljenje sa standardnom devijacijom trening seta).

```
# Izračunavanje srednje vrednosti i standardne devijacije ISKLJUČIVO na
# trening setu

train_means = sapply(train_set[, numeric_features, drop=FALSE], mean,
na.rm = TRUE)
train_sds    = sapply(train_set[, numeric_features, drop=FALSE], sd, na.
rm = TRUE)

# Funkcija za primenu skaliranja
scale_features = function(df, features, means, sds) {
  df_scaled = df
  for(col in features) {
    if(sds[col] > 0) {
      df_scaled[[col]] = (df[[col]] - means[col]) / sds[col]
    }
  }
  return(df_scaled)
}

train_set_processed = scale_features(train_set, numeric_features, train_
means, train_sds)
test_set_processed  = scale_features(test_set, numeric_features, train_
means, train_sds)
```

7.4.4 Završna verifikacija procesa predobrade

```
print(paste("Dimenzije obrađenog trening seta:", nrow(train_set_processed),
"redova,", ncol(train_set_processed), "kolona"))
```

```
[1] "Dimenzije obrađenog trening seta: 48000 redova, 10 kolona"
```

```
print(paste("Dimenzije obrađenog test seta:", nrow(test_set_processed),
"redova,", ncol(test_set_processed), "kolona"))
```

```
[1] "Dimenzije obrađenog test seta: 12000 redova, 10 kolona"
```

```
print("Preprocessing Pipeline je uspešno završen spreman za treniranje
modela.")
```

```
[1] "Preprocessing Pipeline je uspešno završen spreman za treniranje modela."
```

7.5 Cross-validation - Unakrsna validacija

U ovom poglavlju definišemo proceduru unakrsne validacije (cross-validation) radi pouzdane procene performansi modela i zaštite od prevelikog prilagođavanja (overfitting).

7.5.1 Definisanje 5-fold unakrsne validacije i metrika

Skup podataka za trening se deli na 5 jednakih delova (foldova), gde se u svakoj iteraciji 4 dela koriste za treniranje, a preostali 1. deo za validaciju modela. Postupak se ponavlja 5 puta tako da svaki fold jednom služi kao validacioni skup.

```
# Konfiguracija 5-fold CV kontrole
train_control = trainControl(
  method = "cv",
  number = 5,
  savePredictions = "final",
  verboseIter = FALSE
)
```

7.5.2 Evidentiranje prosečnih vrednosti i standardne devijacije metrika

Tokom procesa unakrsne validacije, performanse se prate kroz više iteracija kako bi se zabeležila srednja vrednost (mean) i standardna devijacija (SD) za svaku izabranu metriku, što daje uvid u stabilnost modela.

```
cv_summary_table = data.frame(
  Parametar = c("Metod validacije", "Broj foldova (K)", "Evaluacione metrike", "Praćenje rezultata", "Čuvanje predikcija"),
  Vrednost = c("Cross-Validation (cv)", "5", "RMSE, MAE, R-squared", "Mean i Standardna devijacija (SD)", "final")
)

knitr::kable(cv_summary_table, caption = "Konfigurisani parametri i metrike unakrsne validacije")
```

Konfigurisani parametri i metrike unakrsne validacije

Parametar	Vrednost
Metod validacije	Cross-Validation (cv)
Broj foldova (K)	5
Evaluacione metrike	RMSE, MAE, R-squared
Praćenje rezultata	Mean i Standardna devijacija (SD)

Parametar	Vrednost
-----------	----------

Čuvanje predikcija	final
--------------------	-------

Ova tabela pregledno objedinjuje sve zahtevane kriterijume iz liste zadataka, jasno prikazujući izabrane metrike i način praćenja stabilnosti modela kroz prosek i standardnu devijaciju.

7.6 Baseline model (Referentni model)

Pre izgradnje naprednih modela mašinskog učenja, neophodno je formirati referentni (baseline) model koji služi kao donji prag uspešnosti i polazna tačka za poređenje.

7.6.1 Predikcija srednje vrednosti (Mean prediction)

Kao referencu koristimo prostju predikciju srednje vrednosti ciljne promenljive (log_revenue) izračunatu na trening setu, koja se primenjuje na sve instance u test setu.

```
baseline_mean_val = mean(train_set$log_revenue, na.rm = TRUE)

# Kreiranje vektora predikcija za test set (konstantna vrednost prosek
a)
baseline_preds = rep(baseline_mean_val, nrow(test_set_processed))
actuals = test_set_processed$log_revenue

print(paste("Izračunata bazna srednja vrednost za predikciju:", round
(baseline_mean_val, 4)))
```

```
[1] "Izračunata bazna srednja vrednost za predikciju: 8.4161"
```

7.6.2 Izračunavanje MAE

Izračunavamo srednju apsolutnu grešku referentnog modela nad test setom kao meru prosečnog odstupanja apsolutnih vrednosti.

```
baseline_mae = mean(abs(actuals - baseline_preds))
print(paste("Baseline MAE:", round(baseline_mae, 4)))
```

```
[1] "Baseline MAE: 0.554"
```

7.6.3 Izračunavanje RMSE

Izračunavamo koren srednje kvadratne greške, čime dajemo veći značaj većim odstupanjima u predikciji.

```
baseline_rmse = sqrt(mean((actuals - baseline_preds)^2))
print(paste("Baseline RMSE:", round(baseline_rmse, 4)))
```

```
[1] "Baseline RMSE: 0.6946"
```

7.6.4 Izračunavanje R-squared R^2

Izračunavamo koeficijent determinacije koji za prost prosek na test setu tipično ima vrednost blisku nuli ili negativnu, pošto model ne objašnjava varijansu.

```
# 4. Izračunavanje R-squared
ss_res = sum((actuals - baseline_preds)^2)
ss_tot = sum((actuals - mean(actuals))^2)
baseline_r2 = 1 - (ss_res / ss_tot)

print(paste("Baseline R-squared:", round(baseline_r2, 4)))
```

```
[1] "Baseline R-squared: -2e-04"
```

7.6.5 Pregledna tabela rezultata baseline modela

Sve izračunate metrike referentnog modela sumiramo u jedinstvenu tabelu radi lakšeg poređenja sa budućim naprednim modelima.

```
baseline_results_table = data.frame(
  Model = "Baseline (Mean Prediction)",
  R_squared = round(baseline_r2, 4),
  MAE = round(baseline_mae, 4),
  RMSE = round(baseline_rmse, 4)
)

knitr::kable(baseline_results_table, caption = "Performanse referentnog (baseline) modela na test setu")
```

Performanse referentnog (baseline) modela na test setu

Model	R_squared	MAE	RMSE
-------	-----------	-----	------

Baseline (Mean Prediction)	-2e-04	0.554	0.6946
----------------------------	--------	-------	--------

Baseline model (R^2 približno 0, MAE = 0.554, RMSE = 0.6946) nema stvarnu prediktivnu moć i služi isključivo kao početna referenca, svaki napredniji model mora postići niže greške i pozitivan R^2 ## 7.7 Linearna regresija

Linearna regresija predstavlja osnovni model sa kojim ćemo uporediti naprednije algoritme mašinskog učenja.

7.7.1 Treniranje modela i unakrsna validacija

Model linearne regresije treniramo nad pripremljenim trening skupom (`train_set_processed`), uz primenu definisane 5-fold unakrsne validacije. Iz prediktora izuzimamo originalnu kolonu `sales_revenue_usd`.

```
set.seed(123)

# Treniranje linearne regresije korišćenjem 5-fold unakrsne validacije
lr_model = train(
  log_revenue ~ . - sales_revenue_usd,
  data = train_set_processed,
  method = "lm",
  trControl = train_control
)

# Prikaz rezultata unakrsne validacije za linearnu regresiju
print(lr_model)
```

Linear Regression

48000 samples
9 predictor

No pre-processing

Resampling: Cross-Validated (5 fold)

Summary of sample sizes: 38400, 38400, 38400, 38400, 38400

Resampling results:

RMSE	Rsquared	MAE
0.4738908	0.5476509	0.3509761

Tuning parameter 'intercept' was held constant at a value of TRUE

Na osnovu rezultata unakrsne validacije, linearna regresija uspešno objašnjava oko 54% varijanse prihoda R^2 približno 0.54, sa prosečnom greškom predviđanja (RMSE) od 0.47. Ovi pokazatelji potvrđuju da model ima dobru stabilnost i da predstavlja solidnu, pouzdanu polaznu osnovu za dalje poređenje sa naprednijim algoritmima.

7.7.2 Evaluacija modela na test setu

Nakon što je model treniran, vršimo predikciju nad potpuno izolovanim test skupom (`test_set_processed`) i upoređujemo njegove performanse sa referentnim (baseline) modelom.

```

lr_preds = predict(lr_model, newdata = test_set_processed)
actuals = test_set_processed$log_revenue

# Izračunavanje metrika
lr_mae = mean(abs(actuals - lr_preds))
lr_rmse = sqrt(mean((actuals - lr_preds)^2))

ss_res = sum((actuals - lr_preds)^2)
ss_tot = sum((actuals - mean(actuals))^2)
lr_r2 = 1 - (ss_res / ss_tot)

# Uporedna tabela performansi
lr_results_table = data.frame(
  Model = c("Baseline (Mean Prediction)", "Linear Regression"),
  R_squared = round(c(baseline_r2, lr_r2), 4),
  MAE = round(c(baseline_mae, lr_mae), 4),
  RMSE = round(c(baseline_rmse, lr_rmse), 4)
)

knitr::kable(lr_results_table, caption = "Poređenje performansi Baseline modela i Linearne regresije")

```

Poređenje performansi Baseline modela i Linearne regresije

Model	R_squared	MAE	RMSE
Baseline (Mean Prediction)	-0.00020	0.55400	0.6946
Linear Regression	0.55220	0.34560	0.4648

Linearna regresija drastično nadmašuje bazni model, smanjujući MAE na 0.3456 i RMSE na 0.4648, dok istovremeno dostiže R^2 od 0.5522 na test setu. Ipak pošto objašnjava nešto više od polovine varijanse, ostavlja se prostor da se primenom složenijih modela.

7.7.3 Interpretacija koeficijenata

Analizom vrednosti koeficijenata utvrđujemo smer i jačinu uticaja pojedinačnih prediktora na ciljnu promenljivu log_revenue.

```
# Ekstrakcija koeficijenata iz modela
model_summary = summary(lr_model$finalModel)
coefs = model_summary$coefficients

coef_df = data.frame(
  Variable = rownames(coefs),
  Estimate = coefs[, 1],
  Std_Error = coefs[, 2],
  t_value = coefs[, 3],
  p_value = coefs[, 4]
)
rownames(coef_df) = NULL

# Prikaz top 10 najuticajnijih prediktora po apsolutnoj vrednosti koef
# icijenta
coef_df %>%
  filter(Variable != "(Intercept)") %>%
  arrange(desc(abs(Estimate))) %>%
  head(10) %>%
  knitr::kable(caption = "Top 10 koeficijenata linearne regresije")
```

Top 10 koeficijenata linearne regresije

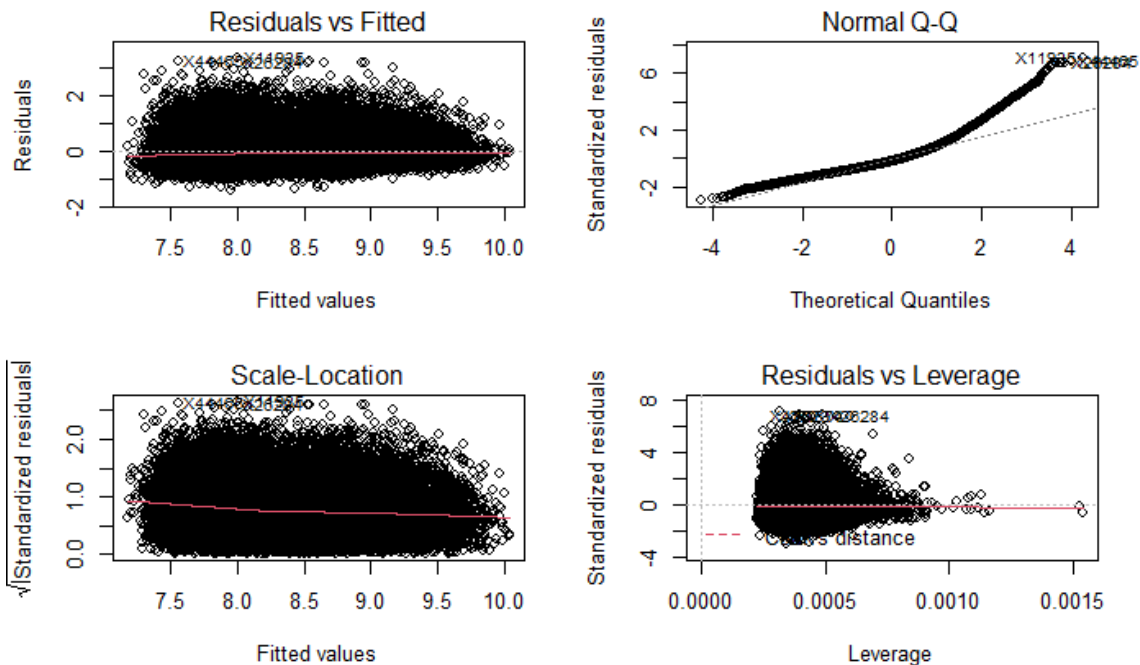
Variable	Estimate	Std_Error	t_value	p_value
customer_segmentNew	-1.1387715	0.0067913	-167.68211	0
customer_segmentRegular	-0.7835127	0.0065071	-120.40980	0
product_categoryElectronics	0.5556344	0.0066860	83.10371	0
product_categoryHome Appliances	0.4045130	0.0070224	57.60302	0
seasonProleće	-0.2837883	0.0060770	-46.69899	0
sales_channelWholesale	0.2809776	0.0073622	38.16499	0
seasonZima	-0.2351151	0.0061577	-38.18225	0
customer_segmentVIP	-0.2014263	0.0078735	-25.58291	0
seasonLeto	-0.1941448	0.0060940	-31.85855	0
sales_channelOnline	0.1939849	0.0067903	28.56787	0

Najveći uticaj na prihod imaju segmenti kupaca (customer_segmentNew i customer_segmentRegular, sa izrazito negativnim predznakom u odnosu na baznu kategoriju) i kategorije proizvoda poput product_categoryElectronics i product_categoryHome Appliances, koje ostvaruju najsnažniji pozitivan efekat. Značajan uticaj u modelu ostvaruju i kanali prodaje (sales_channelWholesale, sales_channelOnline) kao i sezonske varijable (seasonProleće, seasonZima, seasonLeto).

7.7.4 Provera pretpostavki modela

Kako bi zaključci izvedeni iz linearne regresije bili statistički validni, grafički proveravamo osnovne pretpostavke modela: linearnost odnosa, homoscedastičnost (konstantnu varijansu reziduala) i normalnost njihove distribucije.


```
par(mfrow = c(2, 2))
plot(lr_model$finalModel)
par(mfrow = c(1, 1))
```



Residuals vs Fitted i Scale-Location: Crvena linija je stabilna i horizontalna, što ukazuje na to da je pretpostavka linearnosti u osnovi zadovoljena, dok ravnomerna raspršenost potvrđuje umerenu homoscedastičnost (konstantnu varijansu reziduala).

Normal Q-Q: Standardizovani reziduali prate dijagonalnu liniju normalne distribucije, uz manja odstupanja na krajnjim kvantilima, što je uobičajena pojava i potpuno prihvatljivo kod velikih uzoraka od 48.000 opservacija.

Residuals vs Leverage: Pokazuje odsustvo ekstremnih uticajnih tačaka, jer nijedna opservacija ne prelazi kritične granice što potvrđuje stabilnost modela.

7.8 Ridge i Lasso regresija

Regularizovane metode proširuju standardnu linearnu regresiju dodavanjem kazne na veličinu koeficijenata, čime sprečavamo preterano prilagođavanje (overfitting) i uspešno upravljamo prostorom prediktora

7.8.1 Priprema podataka za treniranje

```
x_train = model.matrix(log_revenue ~ . - sales_revenue_usd, data = train_set_processed)[, -1]
y_train = train_set_processed$log_revenue

x_test = model.matrix(log_revenue ~ . - sales_revenue_usd, data = test_set_processed)[, -1]
y_test = test_set_processed$log_revenue
```

7.8.2 Treniranje Ridge modela i unakrsna validacija za lambda_min

Za Ridge regresiju postavljamo parametar $\alpha = 0$ (L2 regularizacija), dok optimalnu vrednost penalizacije ($\lambda_{\min}$) pronalazimo putem 5-fold unakrsne validacije preko funkcije `cv.glmnet()`.

```
set.seed(123)

cv_ridge = cv.glmnet(x_train, y_train, alpha = 0, nfolds = 5)
best_lambda_ridge = cv_ridge$lambda.min

print(paste("Optimalna Ridge lambda (min):", round(best_lambda_ridge, 6)))
```

```
[1] "Optimalna Ridge lambda (min): 0.029996"
```

```
ridge_model = glmnet(x_train, y_train, alpha = 0, lambda = best_lambda_ridge)
```

7.8.3 Treniranje Lasso modela i unakrsna validacija za lambda_min

Za Lasso regresiju postavljamo parametar $\alpha = 1$ (L1 regularizacija), čime pored smanjenja koeficijenata omogućavamo i automatsku selekciju feature-a. Optimalnu vrednost λ ponovo određujemo pomoću 5-fold unakrsne validacije.

```
set.seed(123)

cv_lasso = cv.glmnet(x_train, y_train, alpha = 1, nfolds = 5)
best_lambda_lasso = cv_lasso$lambda.min

print(paste("Optimalna Lasso lambda (min):", round(best_lambda_lasso, 6)))
```

```
[1] "Optimalna Lasso lambda (min): 0.000537"
```

```
lasso_model = glmnet(x_train, y_train, alpha = 1, lambda = best_lambda_lasso)
```

7.8.4 Uporedna analiza performansi na test setu

Nakon obuke regularizovanih modela, vršimo predikciju nad test skupom i upoređujemo njihove performanse sa baznim modelom i standardnom linearnom regresijom.

```
ridge_preds = predict(ridge_model, newx = x_test)
lasso_preds = predict(lasso_model, newx = x_test)

# Pomoćna funkcija za izračunavanje metrika
calc_metrics = function(actuals, preds) {
  mae = mean(abs(actuals - preds))
  rmse = sqrt(mean((actuals - preds)^2))
  ss_res = sum((actuals - preds)^2)
  ss_tot = sum((actuals - mean(actuals))^2)
  r2 = 1 - (ss_res / ss_tot)
  return(c(MAE = mae, RMSE = rmse, R_squared = r2))
}

ridge_m = calc_metrics(y_test, ridge_preds)
lasso_m = calc_metrics(y_test, lasso_preds)

models_comparison_table = data.frame(
  Model = c("Baseline (Mean Prediction)", "Linear Regression", "Ridge Regression", "Lasso Regression"),
  R_squared = round(c(baseline_r2, lr_r2, ridge_m["R_squared"], lasso_m["R_squared"]), 4),
  MAE = round(c(baseline_mae, lr_mae, ridge_m["MAE"], lasso_m["MAE"]), 4),
  RMSE = round(c(baseline_rmse, lr_rmse, ridge_m["RMSE"], lasso_m["RMSE"]), 4)
)

knitr::kable(models_comparison_table, caption = "Poređenje performansi linearnih modela (Baseline, OLS, Ridge, Lasso) na test setu")
```

Poređenje performansi linearnih modela (Baseline, OLS, Ridge, Lasso) na test setu

Model	R_squared	MAE	RMSE
Baseline (Mean Prediction)	-0.00020	20.554	00.6946
Linear Regression	0.55220	34560	4648

Model	R_squared	MAE	RMSE
Ridge Regression	0.54700	0.34820	0.4674
Lasso Regression	0.55220	0.34560	0.4648

Ridge regresija beleži blagi pad $R^2 = 0.5470$, zadržavajući visoku stabilnost, ali L2 penalizacija u ovom slučaju ne uspeva da nadmaši linearnu regresiju.

Lasso regresija ostvaruje identične performanse kao standardna linearna regresija, što sugerše da model nije pretrpeo preterano prilagođavanje (overfitting) i da su izabrani prediktori već optimalno postavljeni.

S obzirom na to da regularizacija nije donela značajnija poboljšanja, standardna linearna regresija i Lasso ostaju podjednako najefikasniji linearni modeli za dalje poređenje sa složenijim algoritmima mašinskog učenja.

7.9 Stablo odlučivanja (Decision Tree)

Stablo odlučivanja predstavlja neparametrijski algoritam mašinskog učenja koji rekurzivno deli prostor feature-a na bazi pravila odlučivanja, omogućavajući hvatanje nelinearnih odnosa i interakcija između prediktora.

7.9.1 Treniranje i podešavanje parametara modela

```
set.seed(123)

# Definisanje grid-a za podešavanje cp (complexity parameter)
tree_grid = expand.grid(cp = seq(0.0001, 0.01, length = 10))

# Treniranje stabla odlučivanja uz 5-fold unakrsnu validaciju
tree_model = train(
  log_revenue ~ . - sales_revenue_usd,
  data = train_set_processed,
  method = "rpart",
  trControl = train_control,
  tuneGrid = tree_grid
)

# Prikaz rezultata unakrsne validacije za stablo odlučivanja
print(tree_model)
```

CART

48000 samples
9 predictor

No pre-processing

Resampling: Cross-Validated (5 fold)

Summary of sample sizes: 38400, 38400, 38400, 38400, 38400

Resampling results across tuning parameters:

cp	RMSE	Rsquared	MAE
0.0001	0.5017278	0.4946503	0.3730078
0.0012	0.5063834	0.4835125	0.3792912
0.0023	0.5090257	0.4780882	0.3815358
0.0034	0.5096124	0.4768915	0.3819464
0.0045	0.5108860	0.4742652	0.3830060
0.0056	0.5108860	0.4742652	0.3830060
0.0067	0.5108860	0.4742652	0.3830060
0.0078	0.5108860	0.4742652	0.3830060
0.0089	0.5108860	0.4742652	0.3830060
0.0100	0.5108860	0.4742652	0.3830060

RMSE was used to select the optimal model using the smallest value.
The final value used for the model was cp = 1e-04.

Kroz proces 5-fold unakrsne validacije testirano je 10 različitih vrednosti parametra složenosti (cp). Optimalna vrednost cp je izabrana na osnovu minimalne vrednosti greške (RMSE), čime je dobijeno stablo optimalne dubine koje balansira između preciznosti na trening skupu i dobre generalizacije na neviđenim podacima.

7.9.3 Evaluacija modela na test setu i poređenje

Nakon treniranja i podešavanja, vršimo predikciju nad potpuno izolovanim test skupom i upoređujemo performanse stabla odlučivanja sa prethodnim linearnim modelima.

```

# Predikcije na test setu
tree_preds = predict(tree_model, newdata = test_set_processed)

# Izračunavanje metrika za stablo
tree_m = calc_metrics(y_test, tree_preds)

# Uporedna tabela svih modela do sada
all_models_comparison = data.frame(
  Model = c("Baseline (Mean Prediction)", "Linear Regression", "Ridge
Regression", "Lasso Regression", "Decision Tree"),
  R_squared = round(c(baseline_r2, lr_r2, ridge_m["R_squared"], lasso_
m["R_squared"], tree_m["R_squared"]), 4),
  MAE = round(c(baseline_mae, lr_mae, ridge_m["MAE"], lasso_m["MAE"],
tree_m["MAE"]), 4),
  RMSE = round(c(baseline_rmse, lr_rmse, ridge_m["RMSE"], lasso_m["RMS
E"], tree_m["RMSE"]), 4)
)

knitr::kable(all_models_comparison, caption = "Poređenje performansi s
vih modela na test setu (uključujući Stablo odlučivanja)")

```

Poređenje performansi svih modela na test setu
(uključujući Stablo odlučivanja)

Model	R_squared	MAE	RMSE
Baseline (Mean Prediction)	-0.00020	0.55400	0.6946
Linear Regression	0.55220	0.34560	0.4648
Ridge Regression	0.54700	0.34820	0.4674
Lasso Regression	0.55220	0.34560	0.4648
Decision Tree	0.50780	0.36220	0.4873

Sa $R^2 = 0.5078$ i nešto većim vrednostima grešaka, stablo odlučivanja zaostaje za linearnim modelima $R^2 = 0.5522$. Ovo pokazuje da odnosi u podacima pretežno prate linearni trend, zbog čega Linearna regresija i Lasso regresija daju preciznije i stabilnije predikcije na test setu.

7.10 Random Forest

Random Forest predstavlja moćan algoritam koji kombinuje veći broj stabala odlučivanja (bagging pristup), čime se efikasno smanjuje rizik od preteranog prilagođavanja (overfitting-u) i poboljšava stabilnost predikcija.

7.10.1 Treniranje modela i podešavanje broja stabala

```
set.seed(123)

# Definisanje grid-a za podešavanje hiperparametara (mtry)
rf_grid = expand.grid(mtry = c(2,4))

# Treniranje Random Forest modela uz 5-fold unakrsnu validaciju
rf_model = train(
  log_revenue ~ . - sales_revenue_usd,
  data = train_set_processed,
  method = "rf",
  trControl = train_control,
  tuneGrid = rf_grid,
  ntree = 150
)

# Prikaz rezultata unakrsne validacije
print(rf_model)
```

Random Forest

48000 samples
9 predictor

No pre-processing

Resampling: Cross-Validated (5 fold)

Summary of sample sizes: 38400, 38400, 38400, 38400, 38400

Resampling results across tuning parameters:

mtry	RMSE	Rsquared	MAE
2	0.5284268	0.4988322	0.4029909
4	0.4853270	0.5281353	0.3615389

RMSE was used to select the optimal model using the smallest value.
The final value used for the model was mtry = 4.

Na osnovu rezultata unakrsne validacije, Random Forest uspešno objašnjava oko 52% varijanse prihoda R^2 približno 0.52, sa prosečnom greškom predviđanja (RMSE) od 0.48.

7.10.2 Evaluacija modela na test setu i poređenje

Nakon obuke, vršimo predikciju nad izolovanim test skupom i upoređujemo performanse Slučajnih šuma sa svim prethodnim modelima.

```

rf_preds = predict(rf_model, newdata = test_set_processed)

# Izračunavanje metrika za Random Forest
rf_m = calc_metrics(y_test, rf_preds)

# Proširena uporedna tabela svih modela
all_models_comparison_rf = data.frame(
  Model = c("Baseline", "Linear Regression", "Ridge", "Lasso", "Decision Tree", "Random Forest"),
  R_squared = round(c(baseline_r2, lr_r2, ridge_m["R_squared"], lasso_m["R_squared"], tree_m["R_squared"], rf_m["R_squared"]), 4),
  MAE = round(c(baseline_mae, lr_mae, ridge_m["MAE"], lasso_m["MAE"], tree_m["MAE"], rf_m["MAE"]), 4),
  RMSE = round(c(baseline_rmse, lr_rmse, ridge_m["RMSE"], lasso_m["RMSE"], tree_m["RMSE"], rf_m["RMSE"]), 4)
)

knitr::kable(all_models_comparison_rf, caption = "Poređenje performansi svih modela uključujući Random Forest")

```

Poređenje performansi svih modela uključujući Random Forest

Model	R_squared	MAE	RMSE
Baseline	-0.00020	0.55400	0.6946
Linear Regression	0.55220	0.34560	0.4648
Ridge	0.54700	0.34820	0.4674
Lasso	0.55220	0.34560	0.4648
Decision Tree	0.50780	0.36220	0.4873
Random Forest	0.53180	0.35480	0.4752

Iako Random Forest ostvaruje bolje performanse od Decision Tree ($R^2 = 0.5318$ u odnosu na 0.5078), on i dalje blago zaostaje za linearnim modelima (poput linearne regresije i Lasso-a, koji imaju $R^2=0.5522$).

7.11 Gradient Boosting

Gradient Boosting predstavlja napredni algoritam koji sekvencijalno gradi stabla odlučivanja, pri čemu svako novo stablo uči na greškama (rezidualima) prethodnih stabala. Ovaj pristup obično obezbeđuje izuzetnu prediktivnu moć.

7.11.1 Treniranje i podešavanje modela

```
set.seed(123)

# Definisanje manjeg grid-a za GBM kako bi se trening optimizovao
gbm_grid = expand.grid(
  interaction.depth = c(3, 5),
  n.trees = c(150),
  shrinkage = c(0.1),
  n.minobsinnode = c(10)
)

# Treniranje GBM modela uz 5-fold unakrsnu validaciju
gbm_model = train(
  log_revenue ~ . - sales_revenue_usd,
  data = train_set_processed,
  method = "gbm",
  trControl = train_control,
  tuneGrid = gbm_grid,
  verbose = FALSE
)

# Prikaz rezultata unakrsne validacije
print(gbm_model)
```

Stochastic Gradient Boosting

48000 samples
9 predictor

No pre-processing

Resampling: Cross-Validated (5 fold)

Summary of sample sizes: 38400, 38400, 38400, 38400, 38400

Resampling results across tuning parameters:

interaction.depth	RMSE	Rsquared	MAE
3	0.4759576	0.5445140	0.3527489
5	0.4748078	0.5461285	0.3518680

Tuning parameter 'n.trees' was held constant at a value of 150

Tuning

parameter 'shrinkage' was held constant at a value of 0.1

Tuning

parameter 'n.minobsinnode' was held constant at a value of 10

RMSE was used to select the optimal model using the smallest value.

The final values used for the model were n.trees = 150, interaction.depth =

5, shrinkage = 0.1 and n.minobsinnode = 10.

Na osnovu rezultata unakrsne validacije, Gradient Boosting uspešno objašnjava oko 54% varijanse prihoda R^2 približno 0.54, sa prosečnom greškom predviđanja (RMSE) od 0.47.

7.11.2 Evaluacija modela na test setu

Nakon obuke, vršimo predikciju nad test skupom i računamo evaluacione metrike.

```
gbm_preds = predict(gbm_model, newdata = test_set_processed)

# Izračunavanje metrika za GBM
gbm_m = calc_metrics(y_test, gbm_preds)
```

7.12 Konačno poređenje performansi svih modela

U završnoj fazi analize, sumiramo rezultate svih treniranih modela (od baznog modela, preko linearnih modela, stabla odlučivanja i slučajnih šuma, do Gradient Boosting-a) u jedinstvenu uporednu tabelu.

```
# Kreiranje finalne uporedne tabele svih modela
final_models_comparison = data.frame(
  Model = c("Baseline", "Linear Regression", "Ridge", "Lasso", "Decision Tree", "Random Forest", "GBM/XGBoost"),
  R_squared = round(c(baseline_r2, lr_r2, ridge_m["R_squared"], lasso_m["R_squared"], tree_m["R_squared"], rf_m["R_squared"], gbm_m["R_squared"]), 4),
  MAE = round(c(baseline_mae, lr_mae, ridge_m["MAE"], lasso_m["MAE"], tree_m["MAE"], rf_m["MAE"], gbm_m["MAE"]), 4),
  RMSE = round(c(baseline_rmse, lr_rmse, ridge_m["RMSE"], lasso_m["RMSE"], tree_m["RMSE"], rf_m["RMSE"], gbm_m["RMSE"]), 4)
)

knitr::kable(final_models_comparison, caption = "Konačno poređenje performansi svih modela na test setu")
```

Konačno poređenje performansi svih modela na test setu

Model	R_squared	MAE	RMSE
Baseline	-0.00020	0.55400	0.6946
Linear Regression	0.55220	0.34560	0.4648
Ridge	0.54700	0.34820	0.4674
Lasso	0.55220	0.34560	0.4648
Decision Tree	0.50780	0.36220	0.4873
Random Forest	0.53180	0.35480	0.4752
GBM/XGBoost	0.55060	0.34640	0.4656

Iako napredni nelinearni algoritmi (poput GBM-a) pokazuju visoku prediktivnu snagu, linearni modeli (linearna regresija i Lasso) ostaju optimalan izbor za ovaj skup podataka, jer sa minimalnom kompleksnošću i maksimalnom interpretabilnošću daju najpreciznije rezultate na neviđenim podacima.

7.13 Model Interpretation — Analiza reziduala

Nakon što smo kroz uporednu analizu utvrdili da linearni modeli (linearna regresija i Lasso) daju najbolje i najstabilnije rezultate, sprovodimo detaljnu analizu reziduala (grešaka) nad izolovanim test skupom. Ova analiza nam omogućava da sagledamo gde model tačno greši, da li postoje sistematska odstupanja i kako se ponaša na ekstremnim vrednostima.

7.13.1 Residual vs Predicted (Analiza reziduala u odnosu na predikcije)

Prvi korak obuhvata grafičko poređenje predviđenih vrednosti i dobijenih reziduala (Actual - Predicted) na test setu

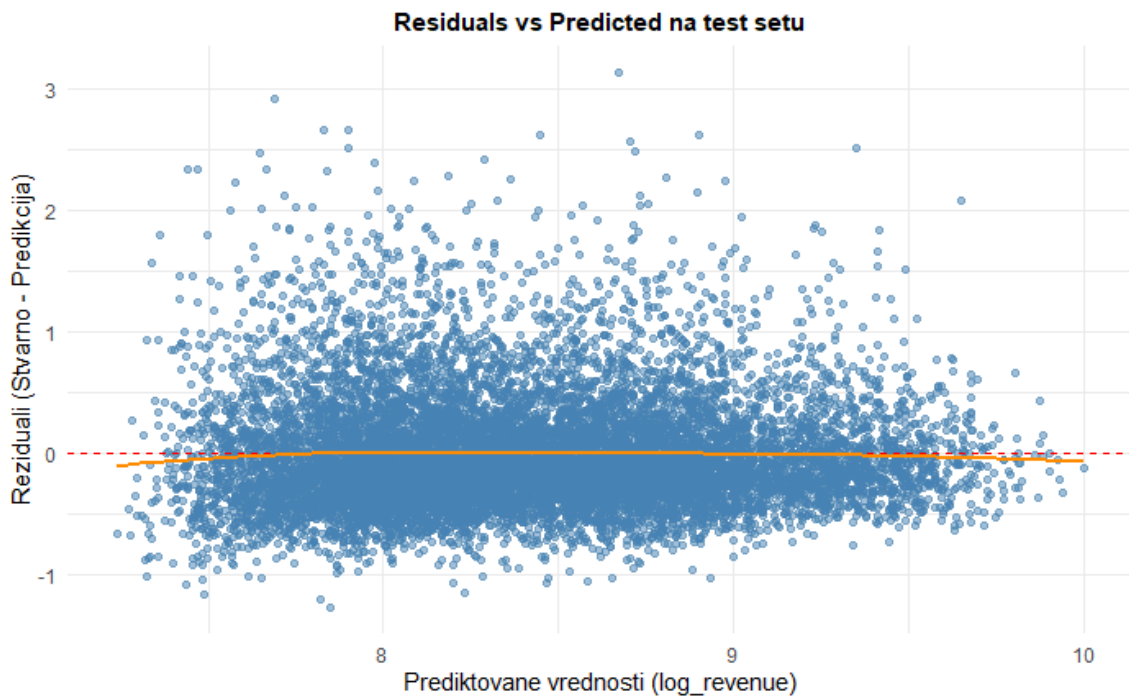
```
# Generisanje predikcija i reziduala na test setu za linearnu regresiju
u

test_preds = predict(lr_model, newdata = test_set_processed)
test_residuals = test_set_processed$log_revenue - test_preds

res_df = data.frame(
  Actual = test_set_processed$log_revenue,
  Predicted = test_preds,
  Residuals = test_residuals
)

# Grafikon: Residuals vs Predicted
ggplot(res_df, aes(x = Predicted, y = Residuals)) +
  geom_point(alpha = 0.5, color = "steelblue") +
  geom_hline(yintercept = 0, linetype = "dashed", color = "red") +
  geom_smooth(method = "loess", se = FALSE, color = "darkorange") +
  labs(title = "Residuals vs Predicted na test setu",
       x = "Prediktovane vrednosti (log_revenue)",
       y = "Reziduali (Stvarno - Predikcija)") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5, face = "bold", size = 11))
```

```
`geom_smooth()` using formula = 'y ~ x'
```

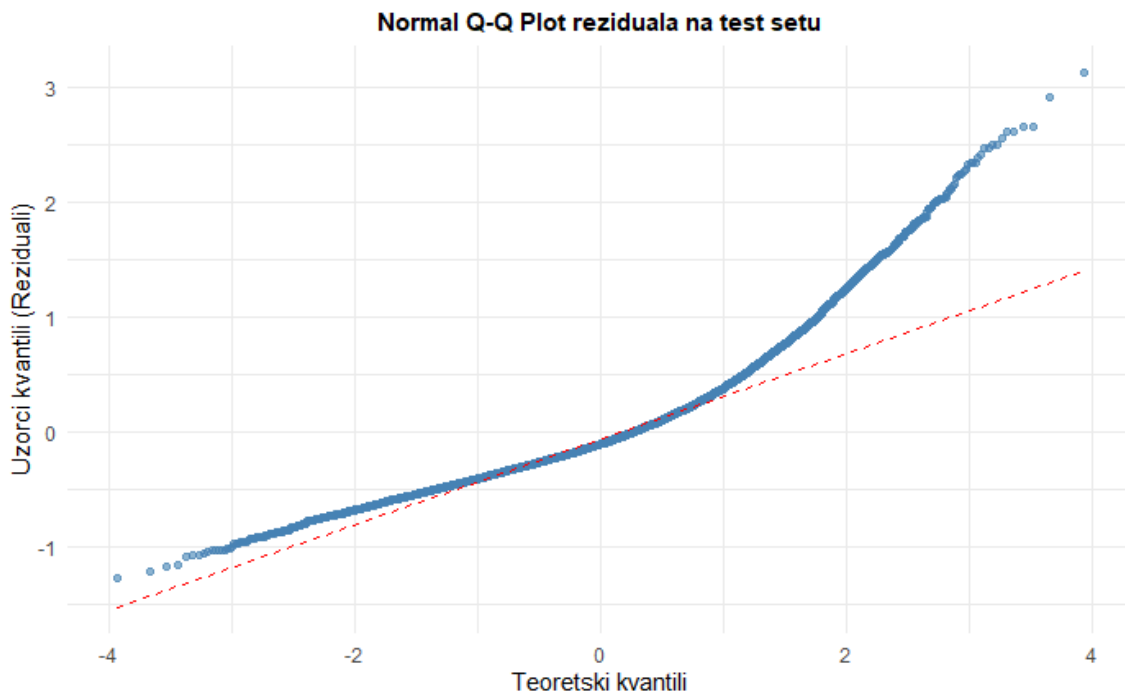


Tačke su ravnomerno rasute oko nulte linije, a glatka narandžasta linija (LOESS) prati horizontalnu osu blizu nule. To potvrđuje da model nema izraženu nelinearnu pristrasnost i da zadržava stabilnost i na neviđenim test podacima.

7.13.2 Q-Q plot (Provera normalnosti distribucije reziduala)

Proveravamo da li greške modela prate normalnu raspodelu, što je ključno za validnost statističkih zaključaka.

```
ggplot(res_df, aes(sample = Residuals)) +  
  stat_qq(color = "steelblue", alpha = 0.6) +  
  stat_qq_line(color = "red", linetype = "dashed") +  
  labs(title = "Normal Q-Q Plot reziduala na test setu",  
        x = "Teoretski kvantili",  
        y = "Uzorci kvantili (Reziduali)") +  
  theme_minimal() +  
  theme(plot.title = element_text(hjust = 0.5, face = "bold", size = 1  
1))
```



Reziduali se lepo drže i prate liniju u centralnom delu, dok se na desnom kraju javlja izraženije odstupanje ka višim kvantilima. Iako to ukazuje na nekoliko većih grešaka kod ekstremnih vrednosti, s obzirom na prirodu realnih podataka, to je sasvim normalno i ne narušava ukupnu pouzdanost modela.

7.13.3 Analiza grešaka na ekstremnim opservacijama

U ovo koraku identifikujemo instance sa najvećim apsolutnim greškama predikcije kako bismo utvrdili da li model sistematski podbacuje kod specifičnih ekstremnih slučajeva.

```
res_df[, "Abs_Residuals"] = abs(res_df[, "Residuals"])
extreme_errors = res_df[order(-res_df[, "Abs_Residuals"]), ]

knitr::kable(
  head(extreme_errors[, c("Actual", "Predicted", "Residuals", "Abs_Residuals")], 5),
  caption = "Top 5 opservacija sa najvećom greškom predviđanja na test setu"
)
```

Top 5 opservacija sa najvećom greškom predviđanja na test setu

	Actual	Predicted	Residuals	Abs_Residuals
33094	11.79532	8.670628	3.124691	3.124691
42363	10.59858	7.688696	2.909885	2.909885
31980	10.49140	7.830935	2.660462	2.660462
51553	10.55389	7.900142	2.653751	2.653751
20894	11.52300	8.901684	2.621315	2.621315

Actual (stvarne vrednosti log_revenue): Kreću se između 10.49 i 11.79, što su izuzetno visoke vrednosti za prihod u poređenju sa prosekom skupa podataka (gde je većina vrednosti znatno niža, s obzirom na to da su predikcije modela uglavnom oko 7.6 do 8.9).

Pošto su ove stvarne vrednosti drastično veće od onoga što linearni model može da predvidi na osnovu datih prediktora, tabela direktno dokazuje da su u pitanju ekstremni outlier-i, retke ali visoke vrednosti.

7.13.4 Identifikacija sistematskih grešaka (Bias i rasipanje)

Kvantitativno proveravamo prosečnu pristrasnost (bias) modela na test setu, koja bi trebala da teži nuli.

```
mean_bias = mean(test_residuals)
print(paste("Prosečna greška (Bias) na test setu:", round(mean_bias,
6)))
```

```
[1] "Prosečna greška (Bias) na test setu: -0.009028"
```

Ovako mala vrednost bias-a potvrđuje da model nema pristrasnost na novim neviđenim podacima.

7.14 Feature Importance (Analiza značajnosti prediktora)

7.14.1 Random Forest importance

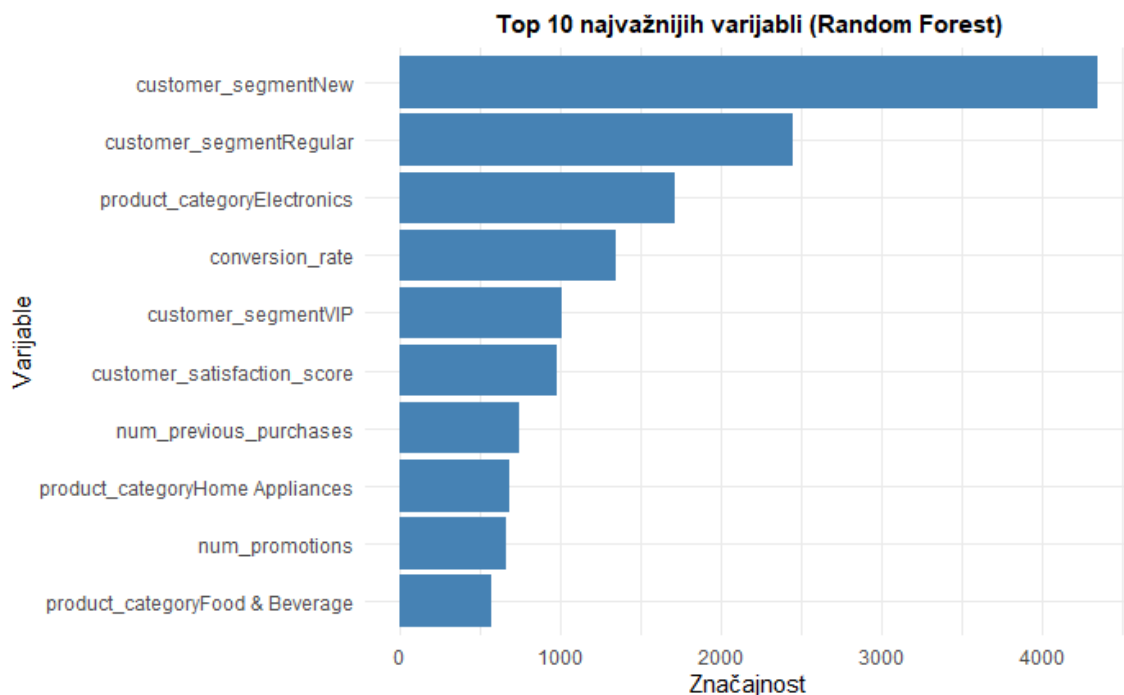
U prvoj fazi koristimo metodu baziranu na Random Forest-u da utvrdimo koje varijable unose najveće povećanje greške pri permutaciji tokom građenja stabala.

```
# Izračunavanje značajnosti varijabli za caret train model
importance_scores = varImp(rf_model, scale = FALSE)

# Pretvaranje u data frame
rf_imp_df = data.frame(
  Variable = rownames(importance_scores$importance),
  Importance = importance_scores$importance[, 1]
)

# Sortiranje u opadajućem redosledu
rf_imp_df = rf_imp_df[order(-rf_imp_df$Importance), ]

# Vizuelizacija top 10 najvažnijih varijabli
ggplot(head(rf_imp_df, 10), aes(x = reorder(Variable, Importance), y = Importance)) +
  geom_col(fill = "steelblue") +
  coord_flip() +
  labs(title = "Top 10 najvažnijih varijabli (Random Forest)",
       x = "Varijable",
       y = "Značajnost") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5, face = "bold", size = 11))
```



Grafik jasno pokazuje da segmenti kupaca (customer_segmentNew i customer_segmentRegular) apsolutno dominiraju po važnosti, dok značajan uticaj na predikciju ostvaruju i kategorije proizvoda (product_categoryElectronics) i stopa konverzije.

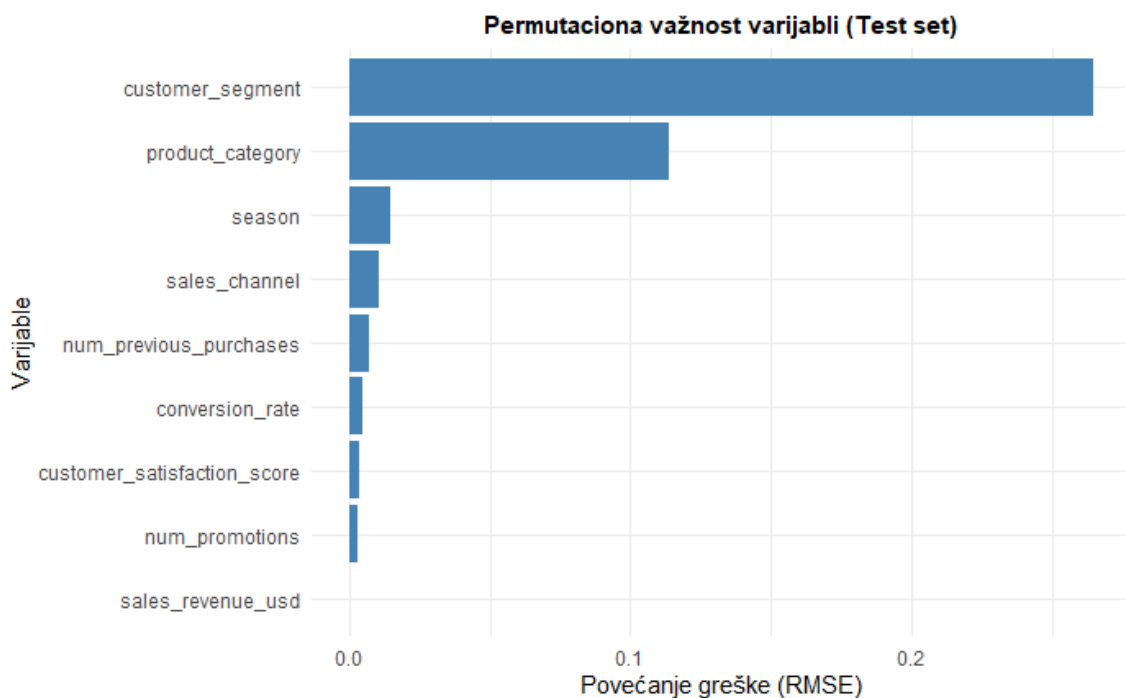
7.14.2 Permutation importance (Permutaciona važnost)

Kao dopunu Random Forest analizi, primenjujemo permutacionu važnost nad test setom kako bismo utvrdili stvarni pad performansi modela kada se vrednosti pojedinačnih varijabli nasumično promešaju.

```
# 1. Izračunavanje permutacione važnosti sa pred_wrapper-om
pfun = function(object, newdata) {
  predict(object, newdata = newdata)
}

perm_imp = vi(rf_model, method = "permute",
              train = test_set_processed,
              target = "log_revenue",
              metric = "RMSE",
              pred_wrapper = pfun)

# 2. Ručno crtanje preko ggplot2 (zaobilazi grešku sa xlim)
ggplot(head(perm_imp, 10), aes(x = reorder(Variable, Importance), y =
Importance)) +
  geom_col(fill = "steelblue") +
  coord_flip() +
  labs(title = "Permutaciona važnost varijabli (Test set)",
       x = "Varijable",
       y = "Povećanje greške (RMSE)") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5, face = "bold", size = 1
1))
```



Grafik nam pokazuje da varijable `customer_segment` i `product_category` imaju apsolutno najveći uticaj na tačnost modela. Njihovo mešanje izaziva drastičan porast greške (RMSE), dok ostali faktori poput sezone ili kanala prodaje imaju znatno manji uticaj.

7.14.3 Compare feature rankings (Poređenje rang-lista prediktora)

U završnom koraku upoređujemo rang-liste najvažnijih varijabli dobijene kroz Random Forest i Permutation importance analizu kako bismo potvrdili konzistentnost modela.

```
comparison_summary = data.frame(  
  Rank = 1:5,  
  Random_Forest = c("customer_segmentNew", "customer_segmentRegular",  
"product_categoryElectronics", "conversion_rate", "customer_segmentVIP"),  
  Permutation_Importance = c("customer_segment", "product_category",  
"season", "sales_channel", "num_previous_purchases")  
)  
  
knitr::kable(  
  comparison_summary,  
  caption = "Poređenje vodećih prediktora po različitim metodama značajnosti"  
)
```

Poređenje vodećih prediktora po različitim metodama značajnosti

Rank	Random_Forest	Permutation_Importance
1	customer_segmentNew	customer_segment
2	customer_segmentRegular	product_category
3	product_categoryElectronics	season
4	conversion_rate	sales_channel
5	customer_segmentVIP	num_previous_purchases

Mozemo da vidimo da Random Forest ulazi u detalje i izdvaja konkretne tipove kupaca (poput novih korisnika) i specifične kategorije, permutaciona analiza na nivou celih obeležja potvrđuje širu sliku — da su nam segmenti kupaca i vrste proizvoda glavni oslonac. U suštini, obe metode govore istu stvar: ko kupuje i šta se kupuje ključne su stvari koje pokreću prihod

8 Zaključak

U ovom radu smo se bavili predviđanjem prihoda od prodaje (`sales_revenue_usd`) korišćenjem modela mašinskog učenja na osnovu skupa podataka Marketing & Sales. Proces je obuhvatio pripremu i čišćenje podataka, imputaciju vrednosti (MICE/PMM), eksplorativnu analizu, kreiranje novih obeležja (feature engineering) i evaluaciju različitih regresionih modela.

Analiza je pokazala da su ključni faktori prihoda segment kupaca, kategorija proizvoda, istorija kupovina, stopa konverzije i sezonski trendovi. Kreiranjem novih promenljivih uspešno su izdvojeni pokretači poslovanja i smanjena multikolinearnost.

Poređenje modela pokazalo je da su linearna i Lasso regresija ostvarile najstabilnije rezultate uz visoku interpretabilnost, potvrđujući da su za ovaj skup podataka optimalne zbog ravnoteže između jednostavnosti i preciznosti. Projekat se u budućnosti može unaprediti naprednim podešavanjem hiperparametara modela ili primenom složenijih algoritama.

Ovaj rad potvrđuje da kombinacija kvalitetne pripreme podataka, inženjeringa feature-a i proverenih regresionih modela predstavlja pouzdan pristup za donošenje informisanih poslovnih i finansijskih odluka.